

Finding a Small Vertex Cut on Distributed Networks

Yonggang Jiang, Sagnik Mukhopadhyay



The
University
Of
Sheffield.

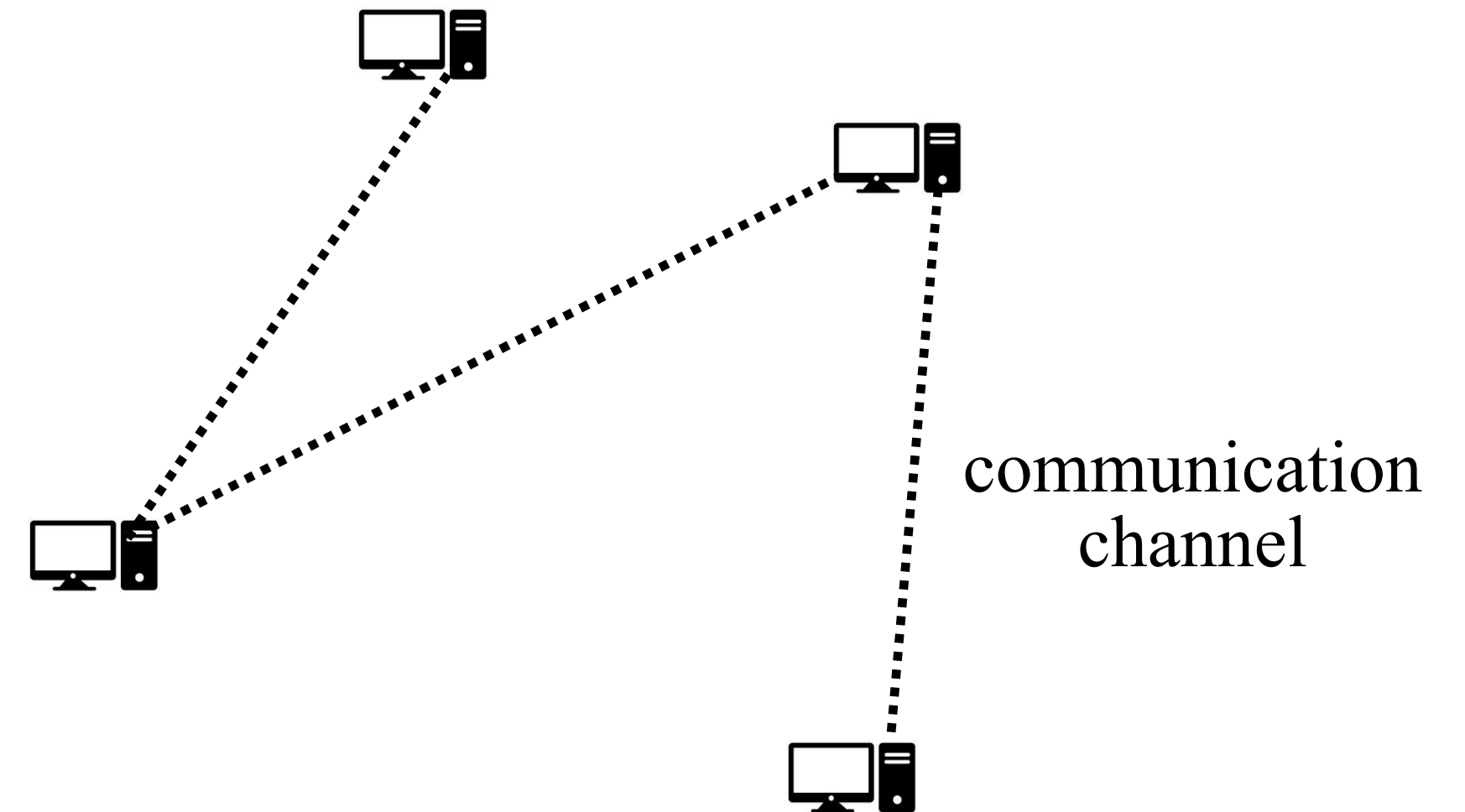
CONGEST Model

n computers



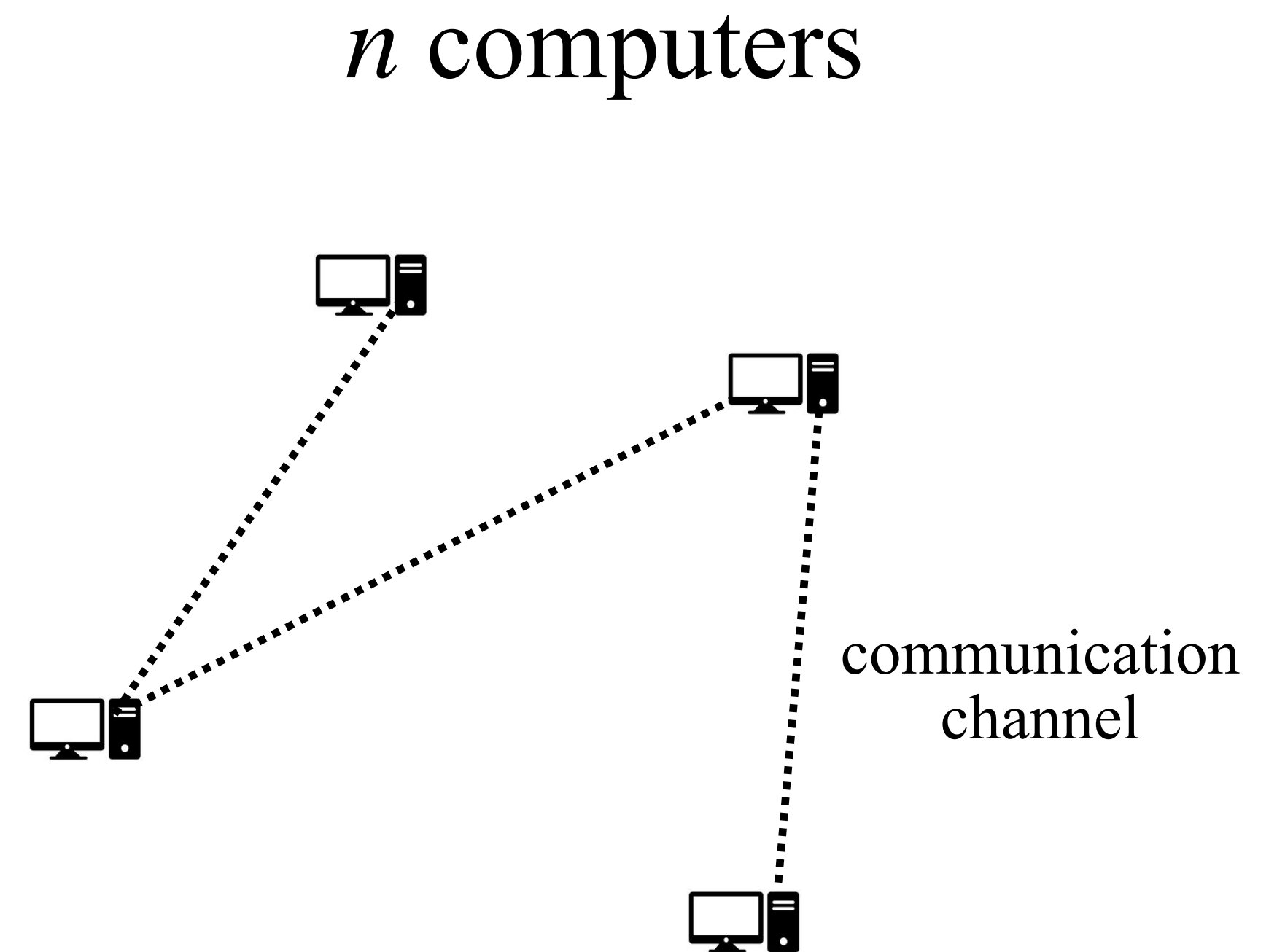
CONGEST Model

n computers



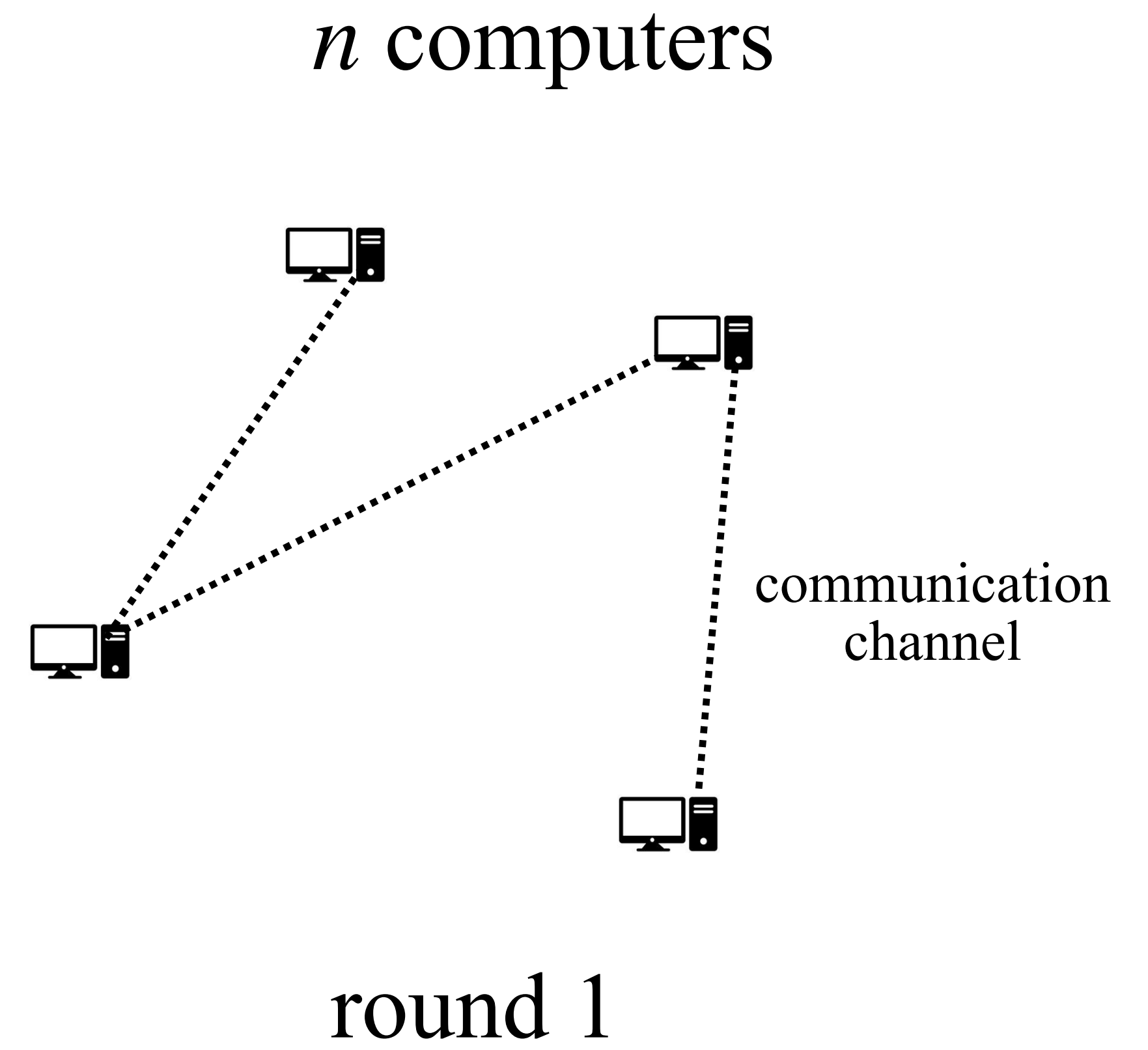
CONGEST Model

- Communication happens in **synchronised** rounds.



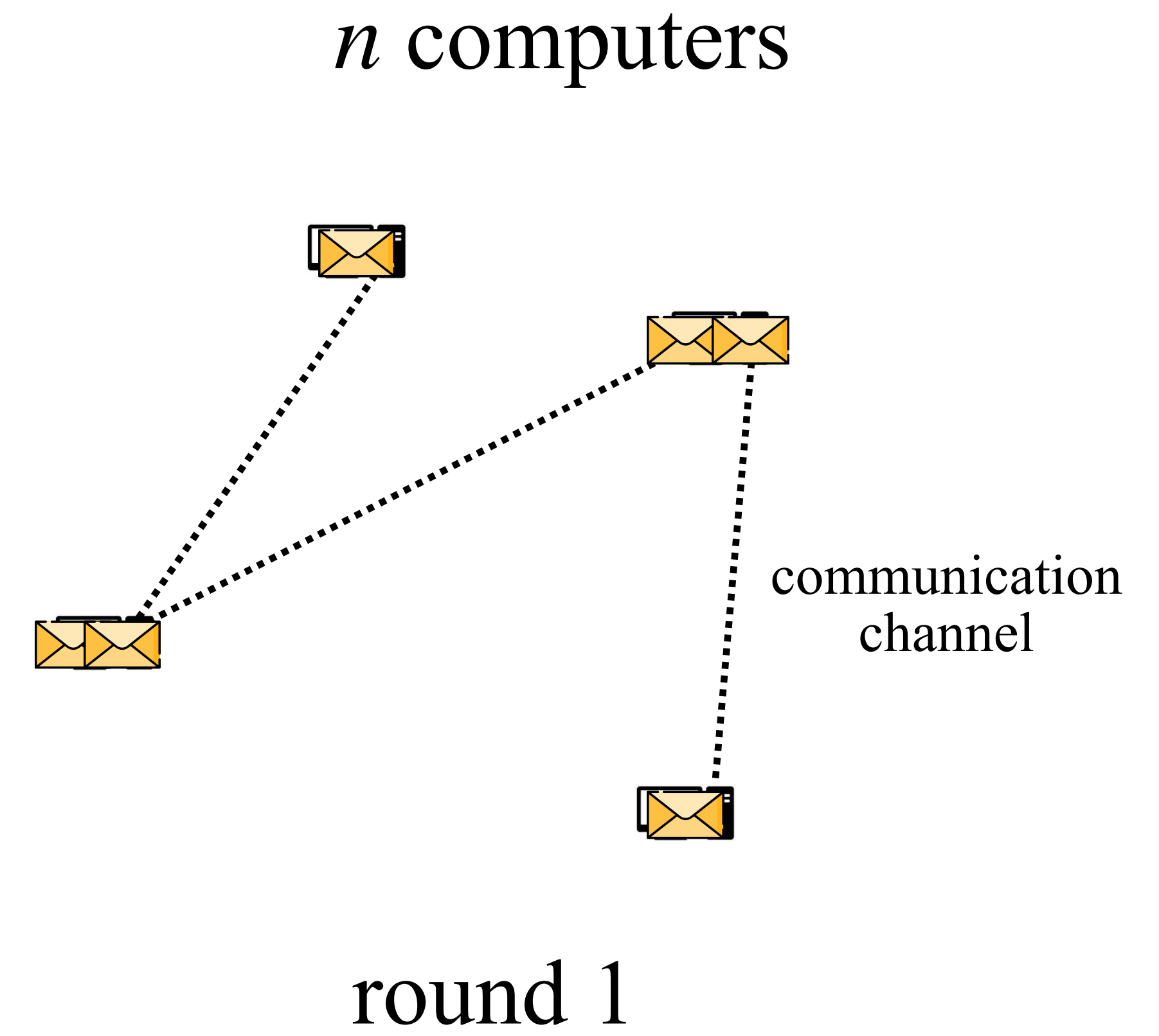
CONGEST Model

- Communication happens in **synchronised** rounds.



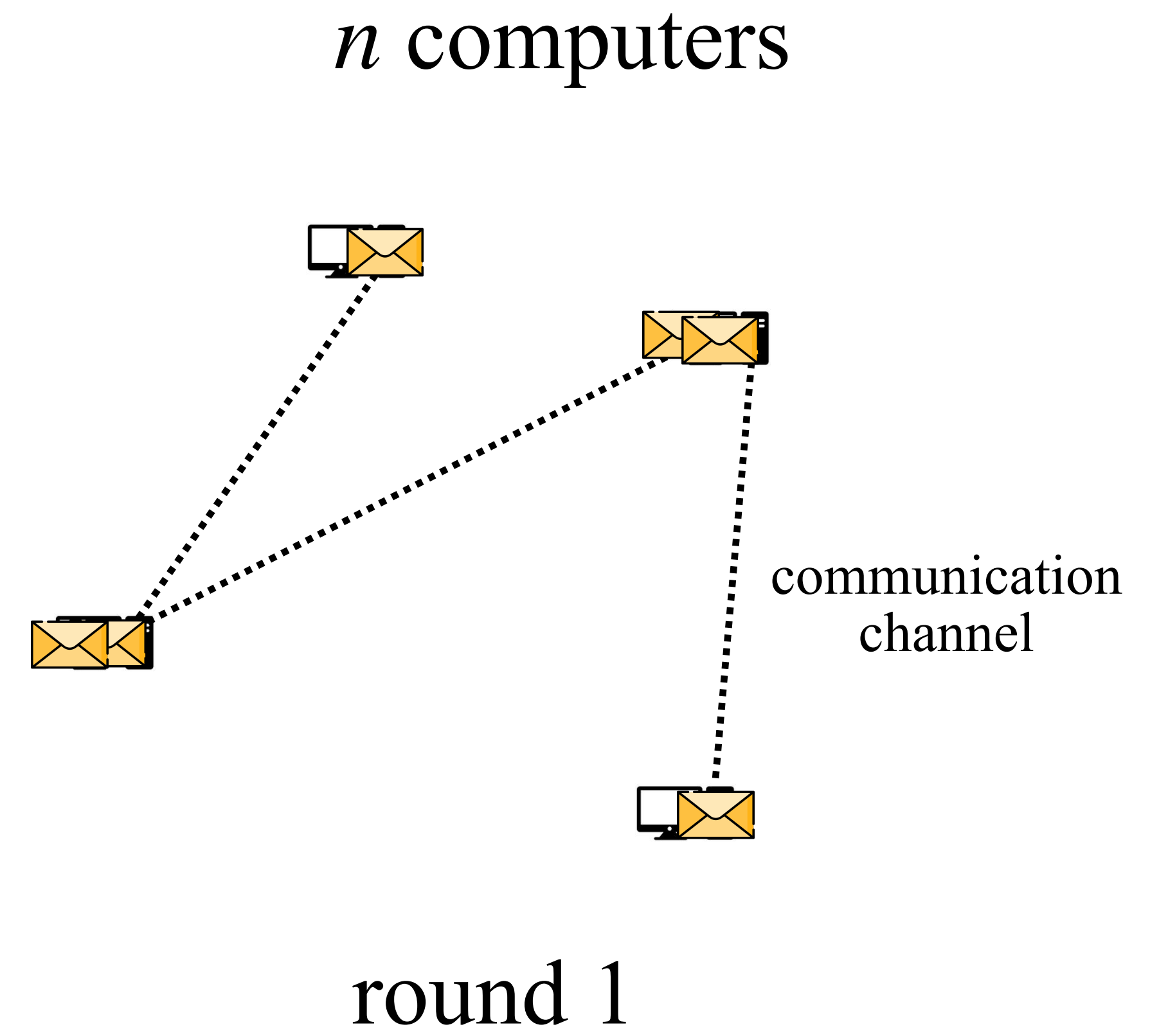
CONGEST Model

- Communication happens in **synchronised** rounds.



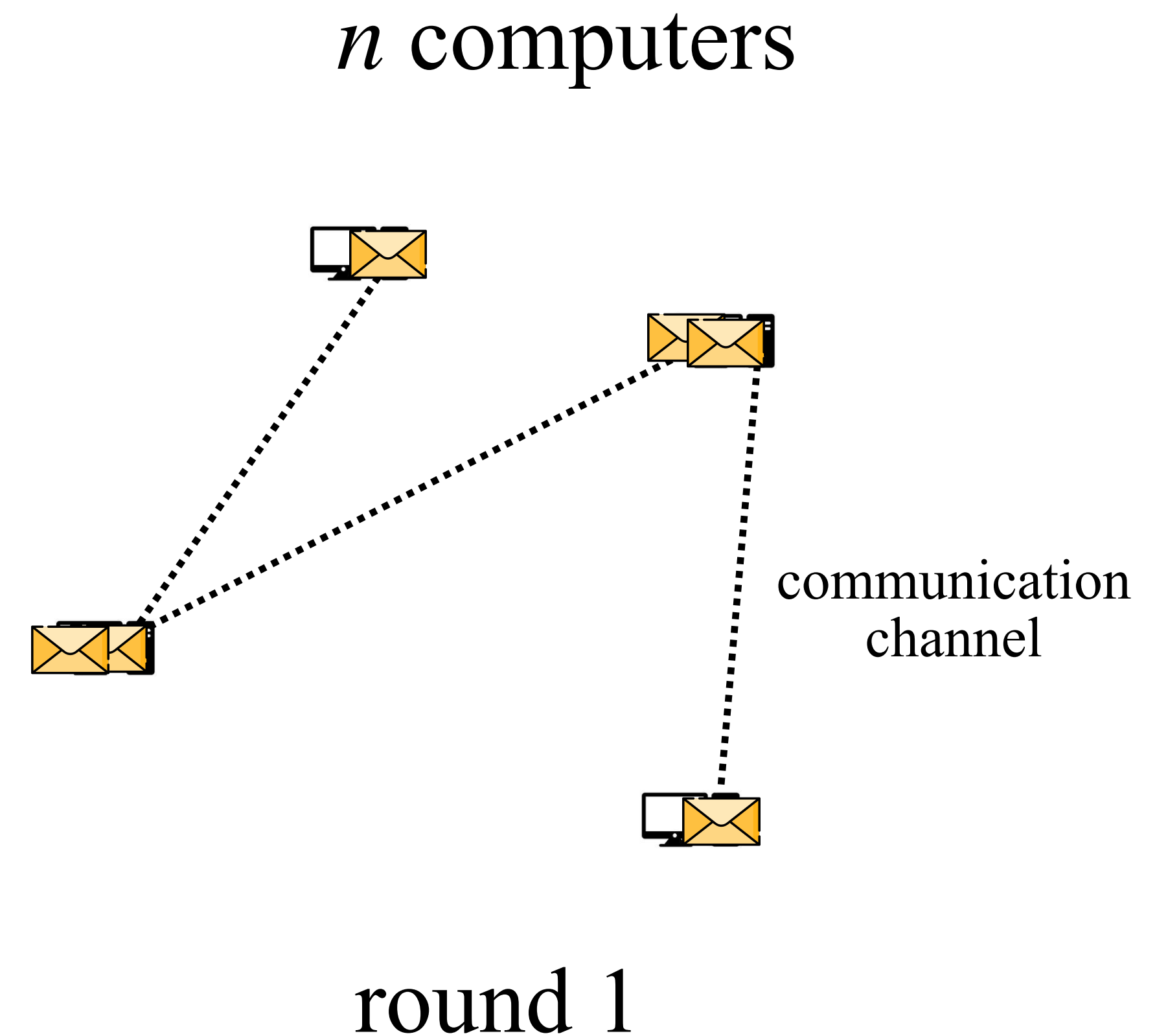
CONGEST Model

- Communication happens in **synchronised** rounds.



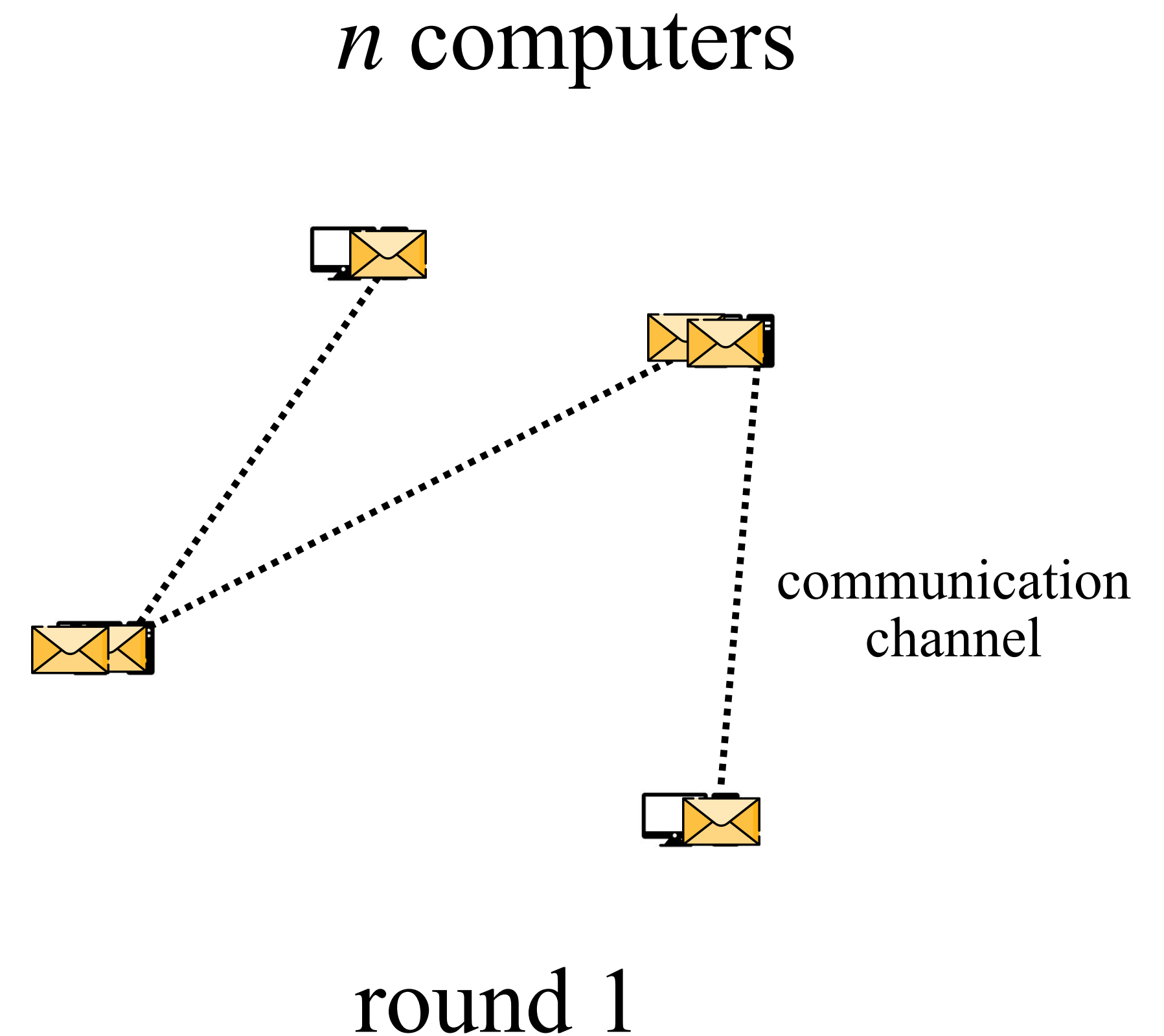
CONGEST Model

- Communication happens in **synchronised** rounds.
- Message size = $O(\log n)$.



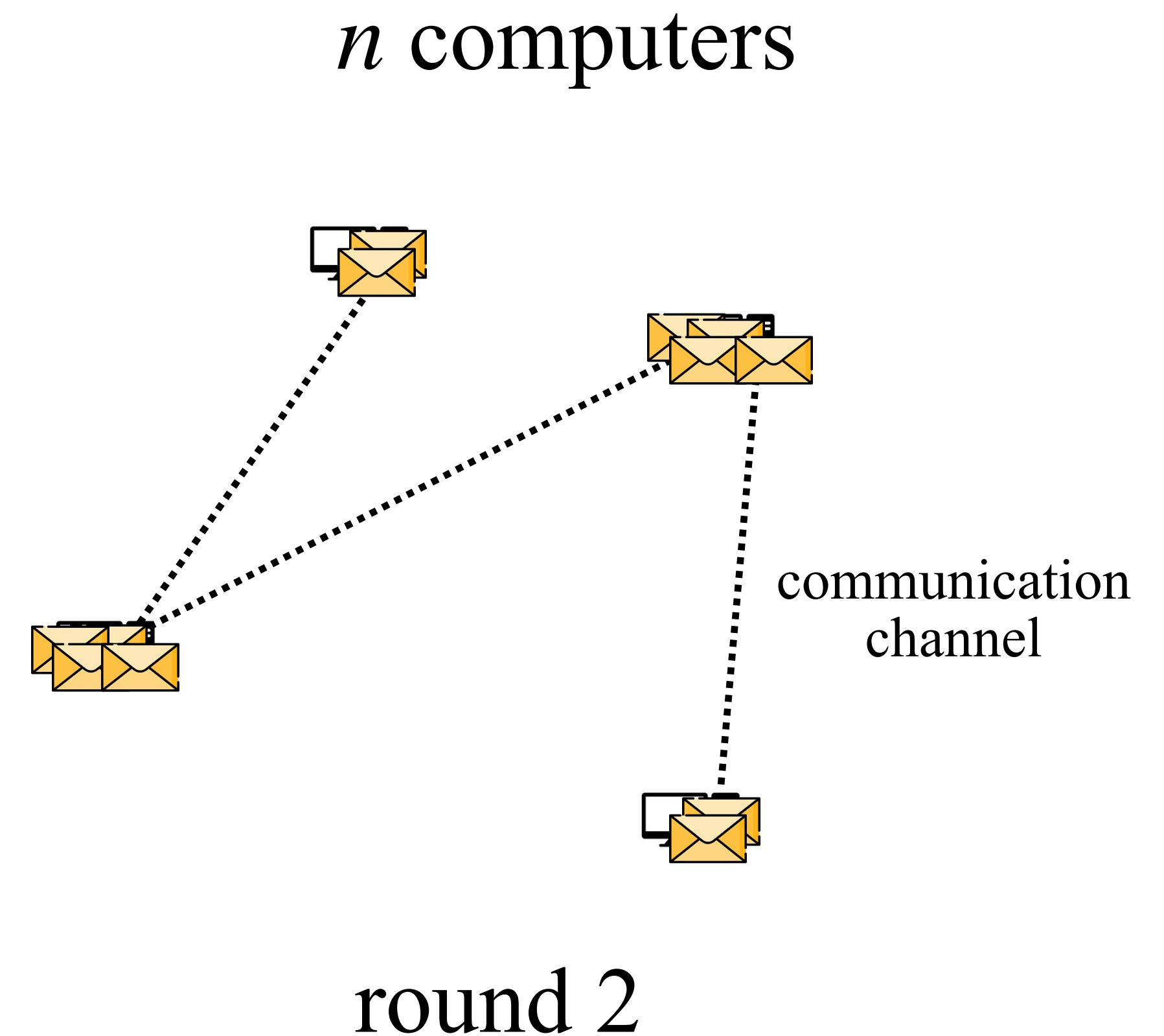
CONGEST Model

- Communication happens in **synchronised** rounds.
- Message size = $O(\log n)$.
- **Unlimited local** computation at the end of each round.



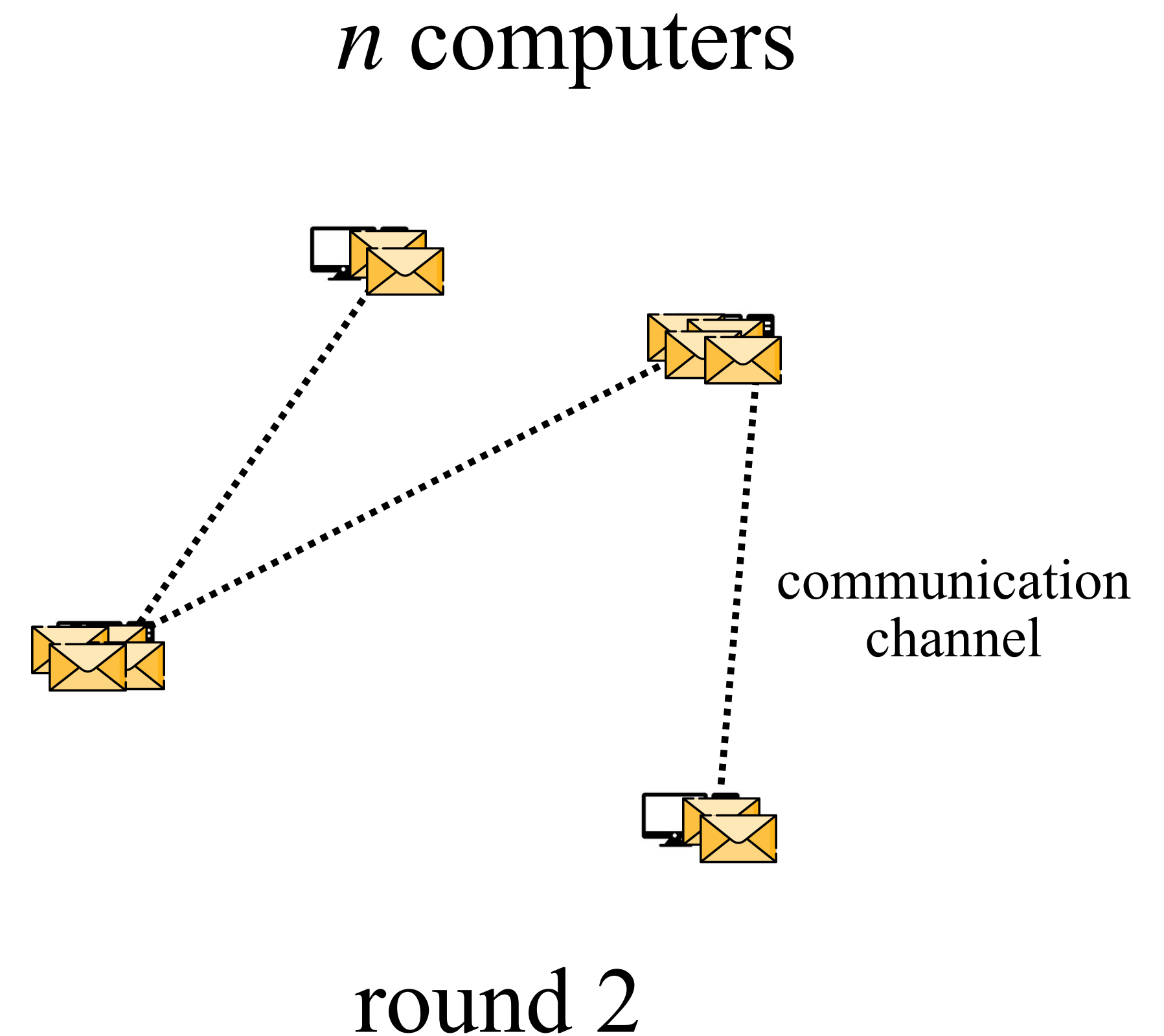
CONGEST Model

- Communication happens in **synchronised** rounds.
- Message size = $O(\log n)$.
- **Unlimited local** computation at the end of each round.



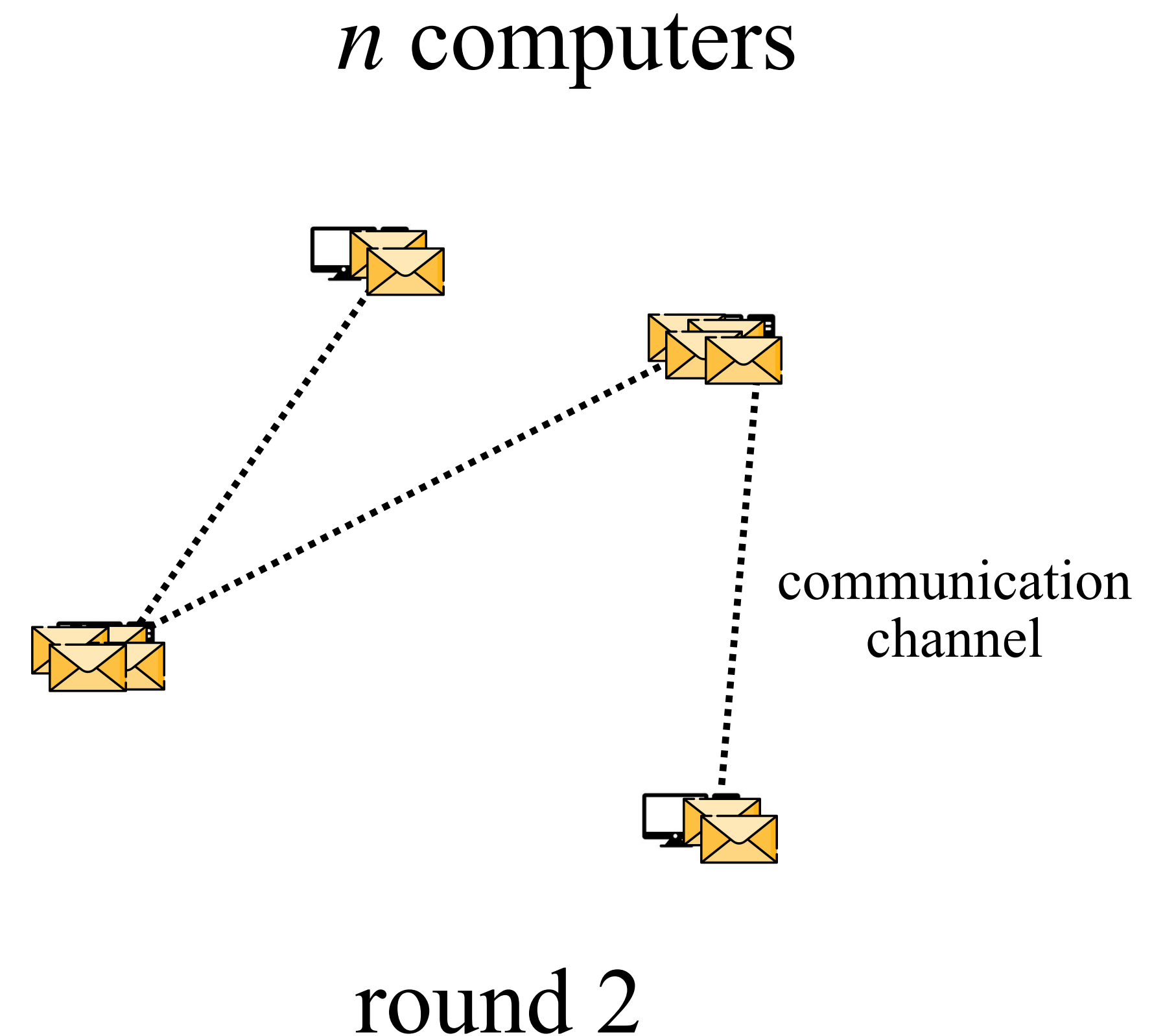
CONGEST Model

- Communication happens in **synchronised** rounds.
- Message size = $O(\log n)$.
- **Unlimited local** computation at the end of each round.



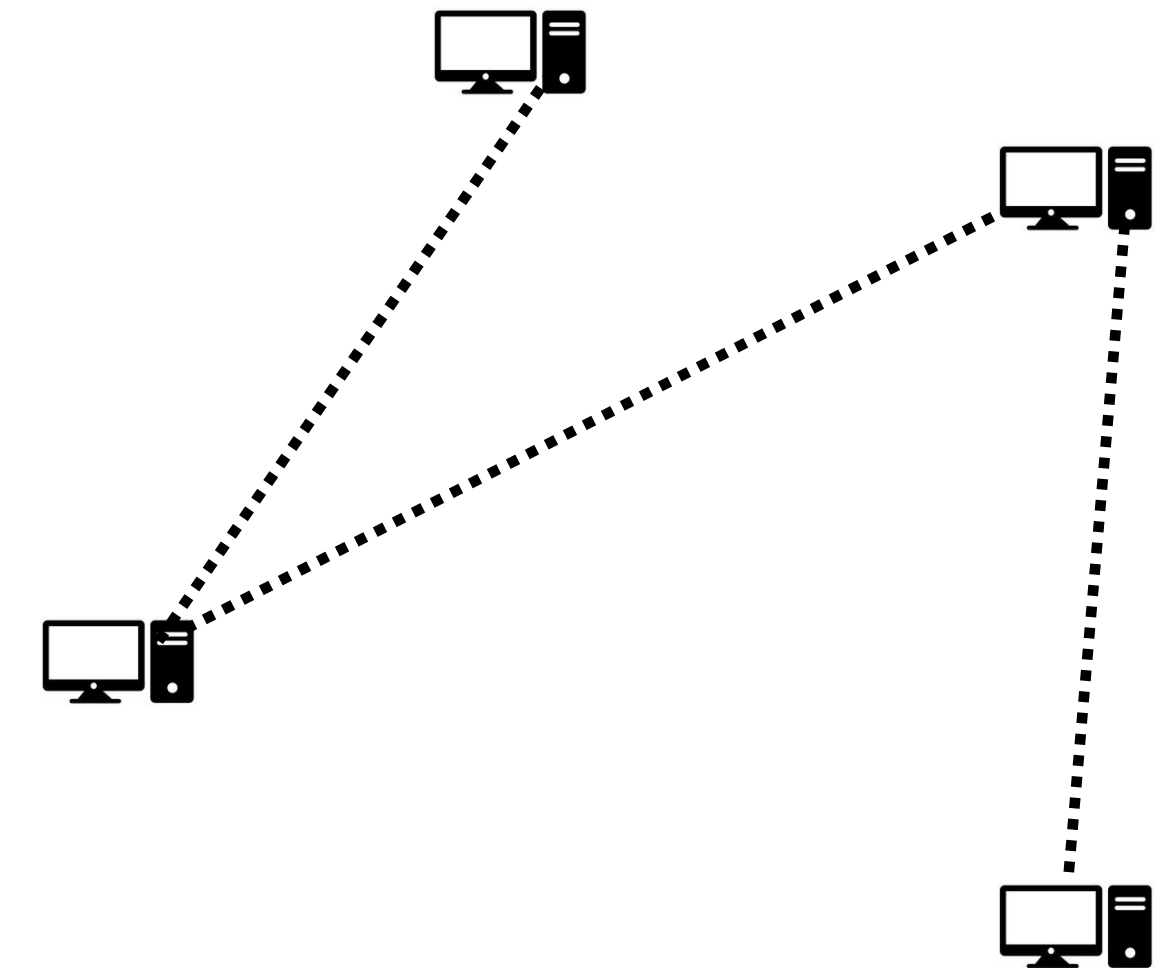
CONGEST Model

- Communication happens in **synchronised** rounds.
- Message size = $O(\log n)$.
- **Unlimited local** computation at the end of each round.
- Complexity measurement: number of rounds.



Vertex Cut

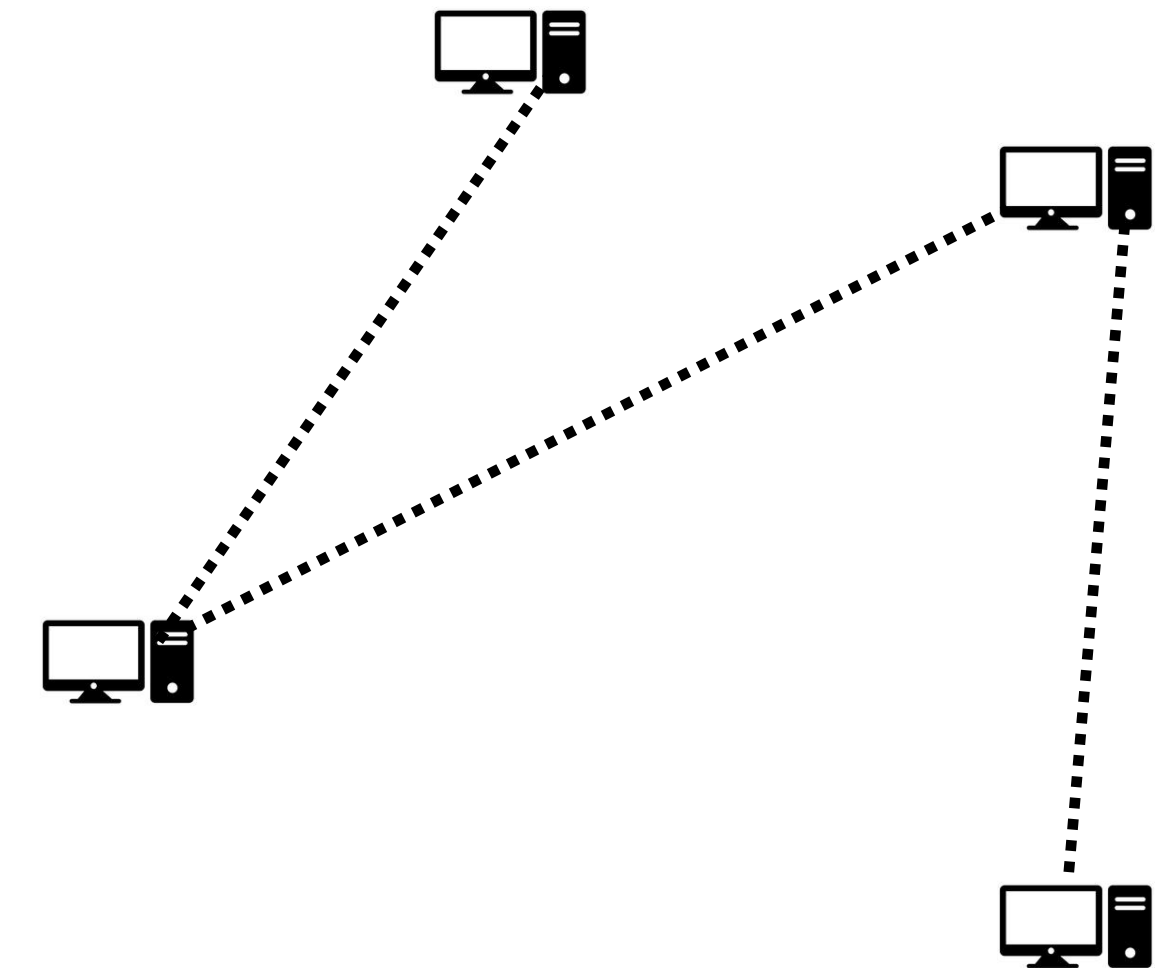
Undirected graph $G = (V, E)$



Vertex Cut

- $S \subseteq V$ is a **vertex cut** if removing S disconnects the graph.

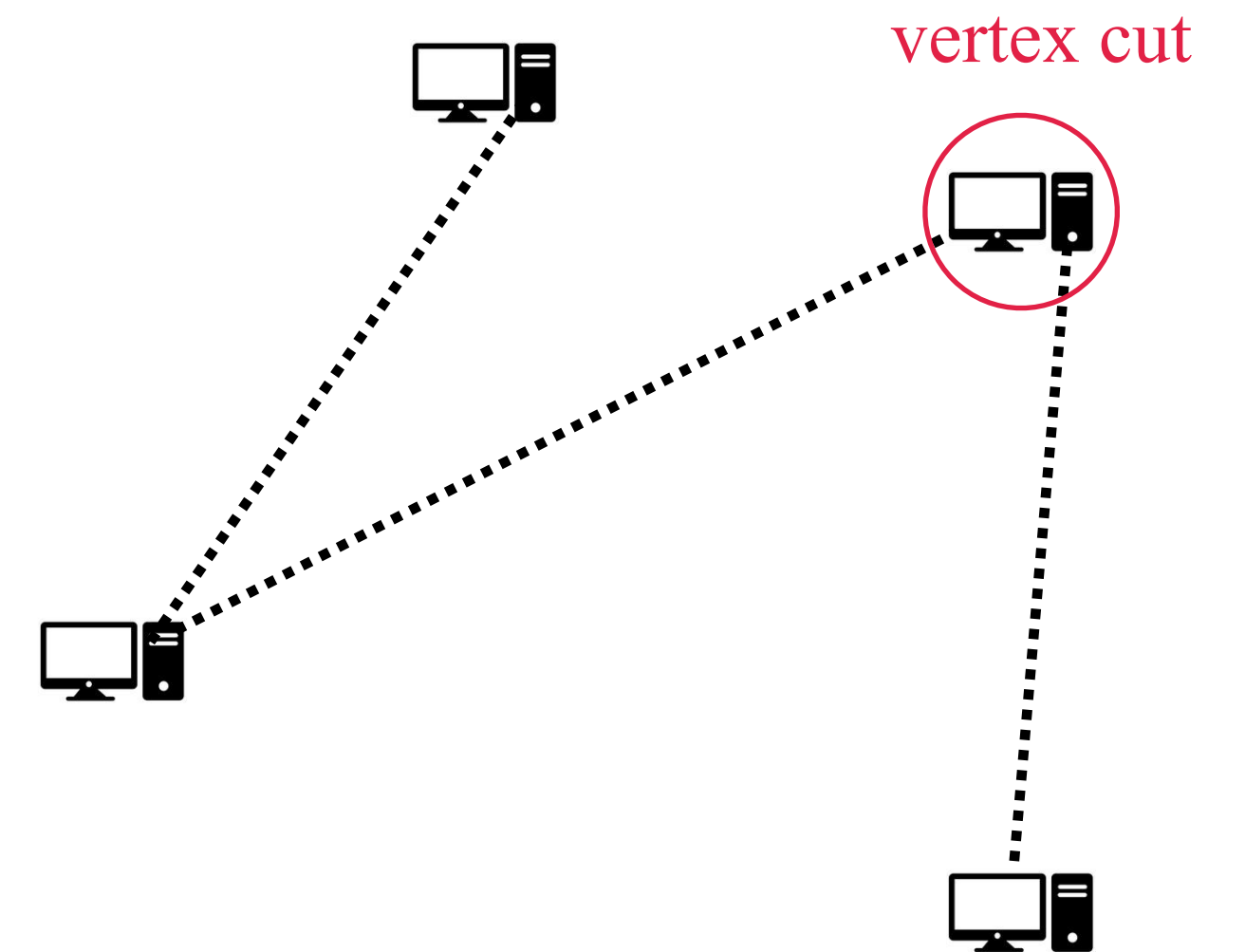
Undirected graph $G = (V, E)$



Vertex Cut

- $S \subseteq V$ is a **vertex cut** if removing S disconnects the graph.

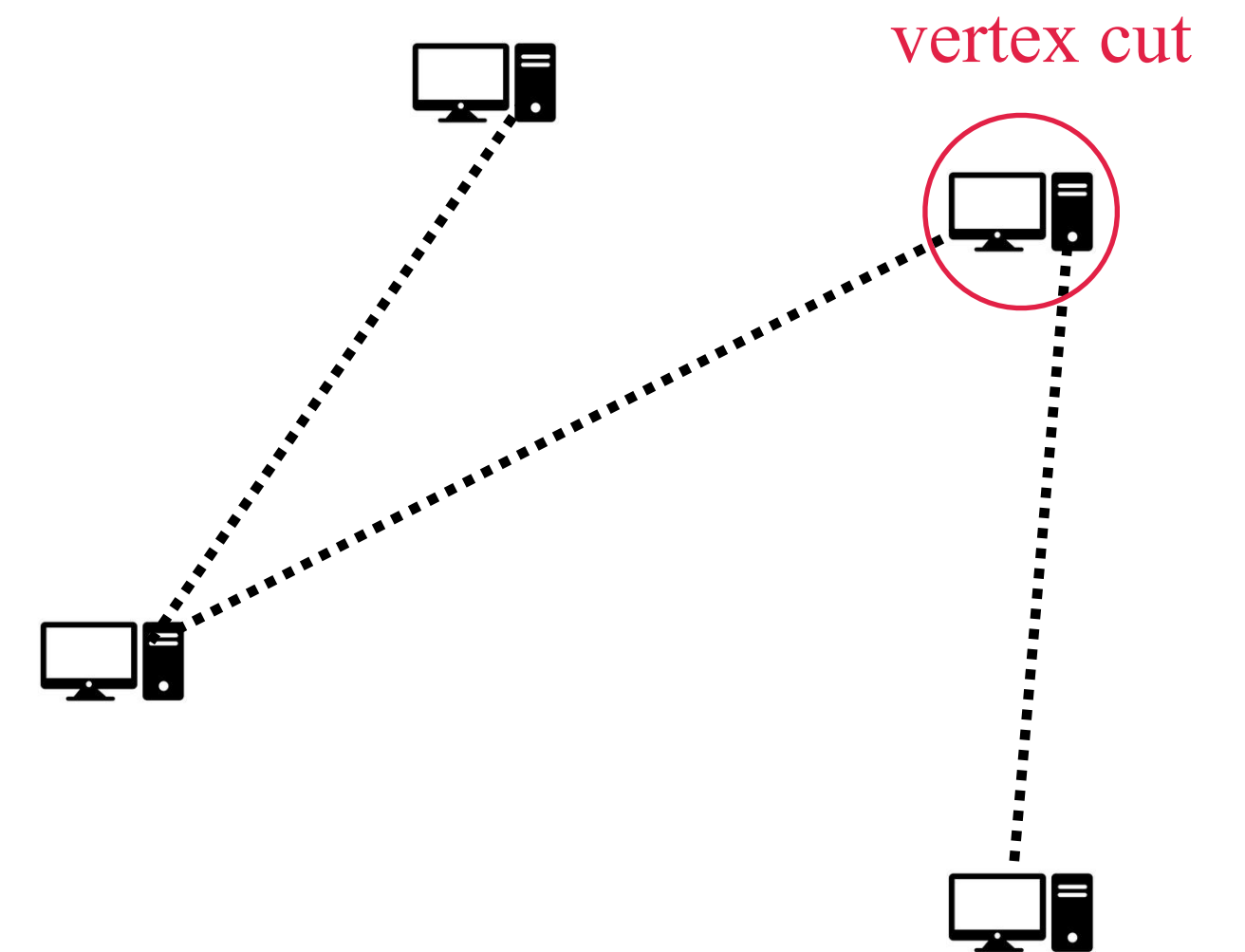
Undirected graph $G = (V, E)$



Vertex Cut

- $S \subseteq V$ is a **vertex cut** if removing S disconnects the graph.
- Problem definition (**vertex connectivity**): find a smallest vertex cut.

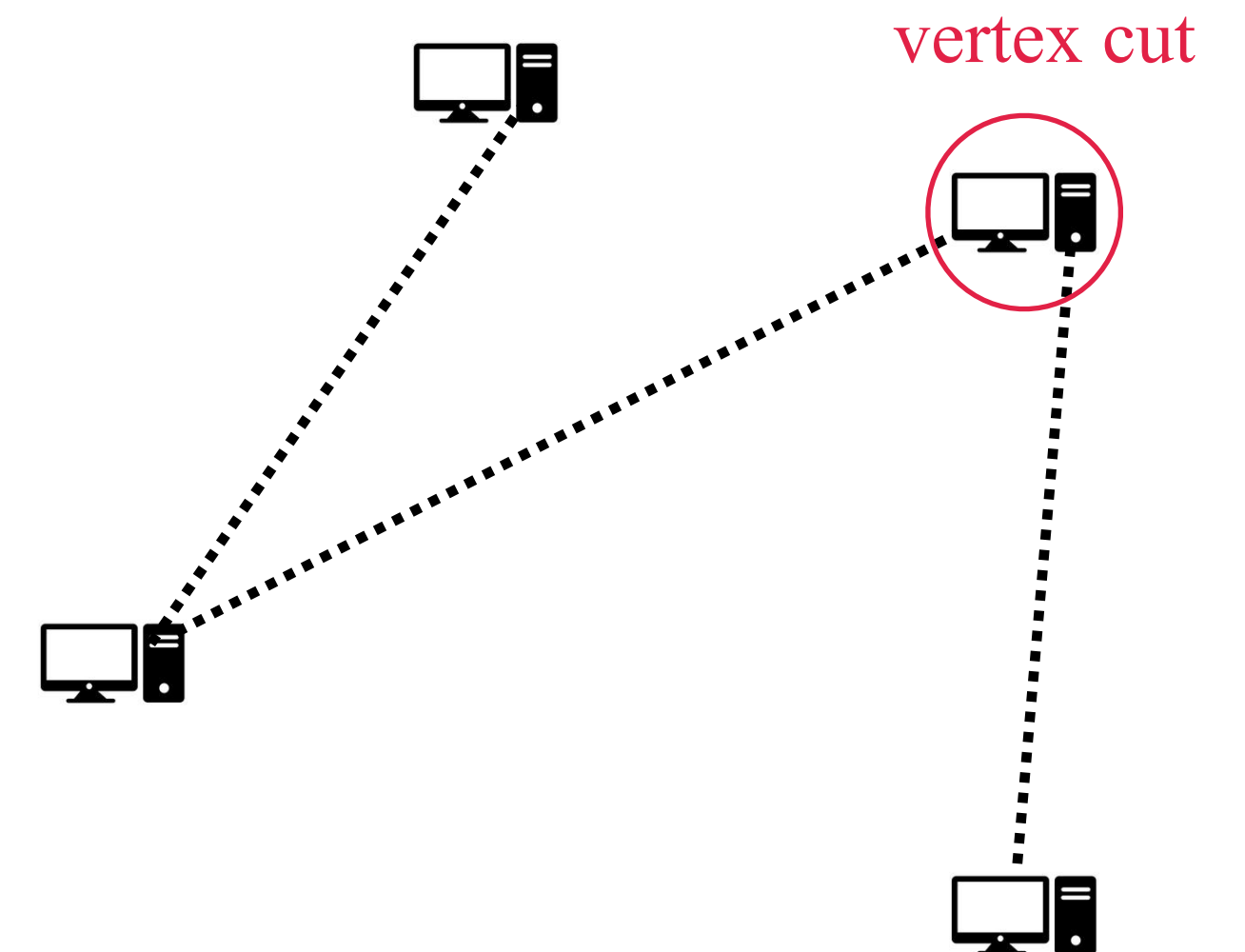
Undirected graph $G = (V, E)$



Vertex Cut

- $S \subseteq V$ is a **vertex cut** if removing S disconnects the graph.
- Problem definition (**vertex connectivity**): find a smallest vertex cut.
- We use k to denote the size of the smallest vertex cut.

Undirected graph $G = (V, E)$



We use k to denote the size of the smallest vertex cut.

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

Related Works: CONGEST Model

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

Related Works: CONGEST Model

- [Censor-Hillel, Ghaffari, Kuhn, 2014] $O(\log n)$ approximation in $\tilde{O}(D + \sqrt{n})$ rounds

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

Related Works: CONGEST Model

- [Censor-Hillel, Ghaffari, Kuhn, 2014] $O(\log n)$ approximation in $\tilde{O}(D + \sqrt{n})$ rounds
- [Parter, Petruschka, 2022] exact in $\tilde{O}(D)$ for $k = 1, 2$

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

Related Works: CONGEST Model

- [Censor-Hillel, Ghaffari, Kuhn, 2014] $O(\log n)$ approximation in $\tilde{O}(D + \sqrt{n})$ rounds
- [Parter, Petruschka, 2022] exact in $\tilde{O}(D)$ for $k = 1, 2$

No sub-linear algorithm exists for $k \geq 3$!

We use k to denote the size of the smallest vertex cut.

Related Works: Sequential Model

- [Forster, Nanongkai, Yang, Saranurak, Yingchareonthawornchai, 2020] $\tilde{O}(k^2 m)$
- [Li, Nanongkai, Panigrahi, Saranurak, Yingchareonthawornchai, 2021] $\tilde{O}(m^{1+o(1)})$

Related Works: CONGEST Model

- [Censor-Hillel, Ghaffari, Kuhn, 2014] $O(\log n)$ approximation in $\tilde{O}(D + \sqrt{n})$ rounds
- [Parter, Petruschka, 2022] exact in $\tilde{O}(D)$ for $k = 1, 2$
- This work: exact in $k^3 \cdot \tilde{O}(\sqrt{n} + D)$

Open Problems

Open Problems

- Better than $\tilde{O}(\sqrt{n} + D)$ algorithm for small k .

Open Problems

- Better than $\tilde{O}(\sqrt{n} + D)$ algorithm for small k .
- Sub-linear round algorithm for arbitrary k .

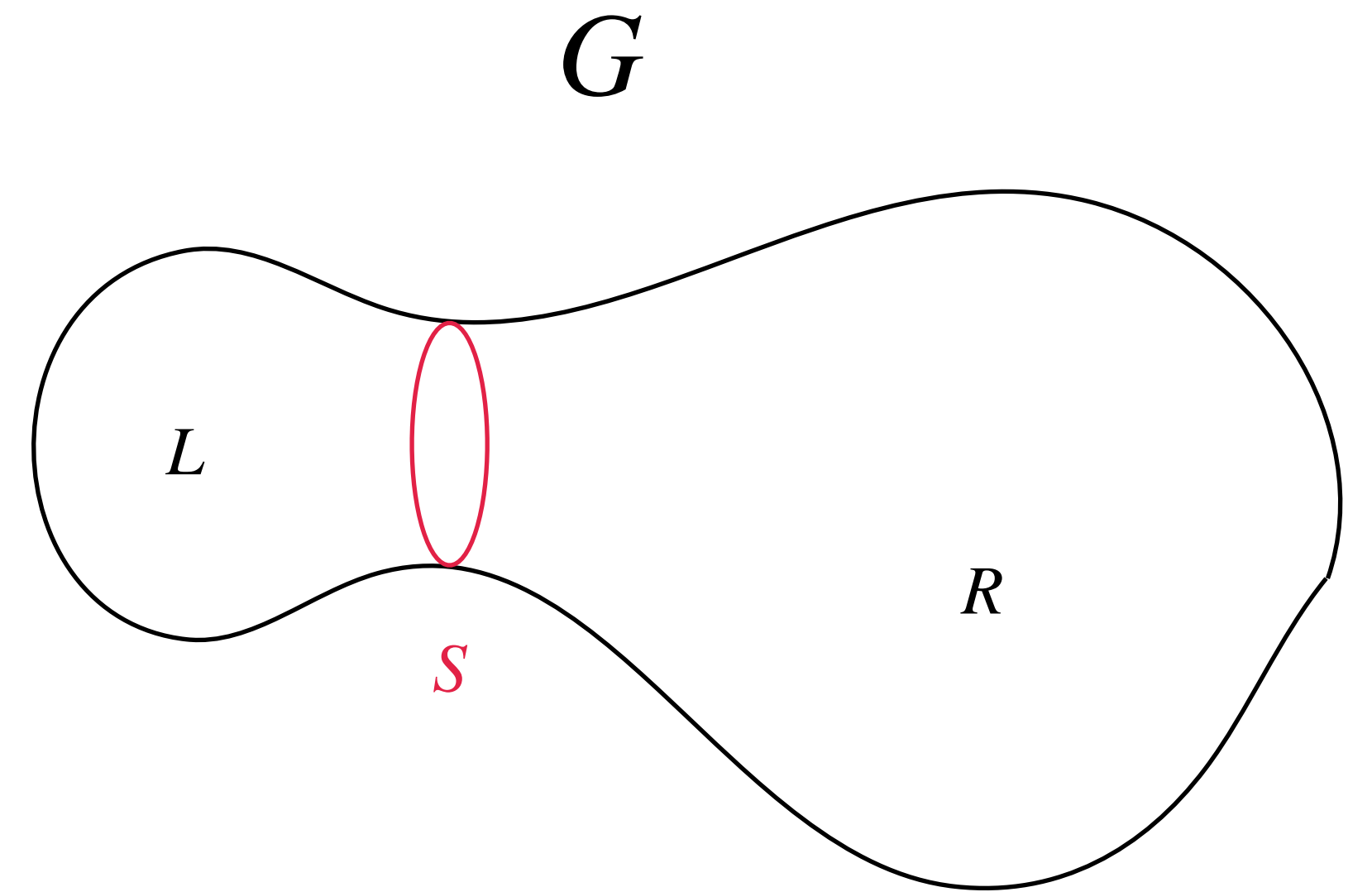
Open Problems

- Better than $\tilde{O}(\sqrt{n} + D)$ algorithm for small k .
- Sub-linear round algorithm for arbitrary k .
- Vertex connectivity in other models: parallel, ...

Outline

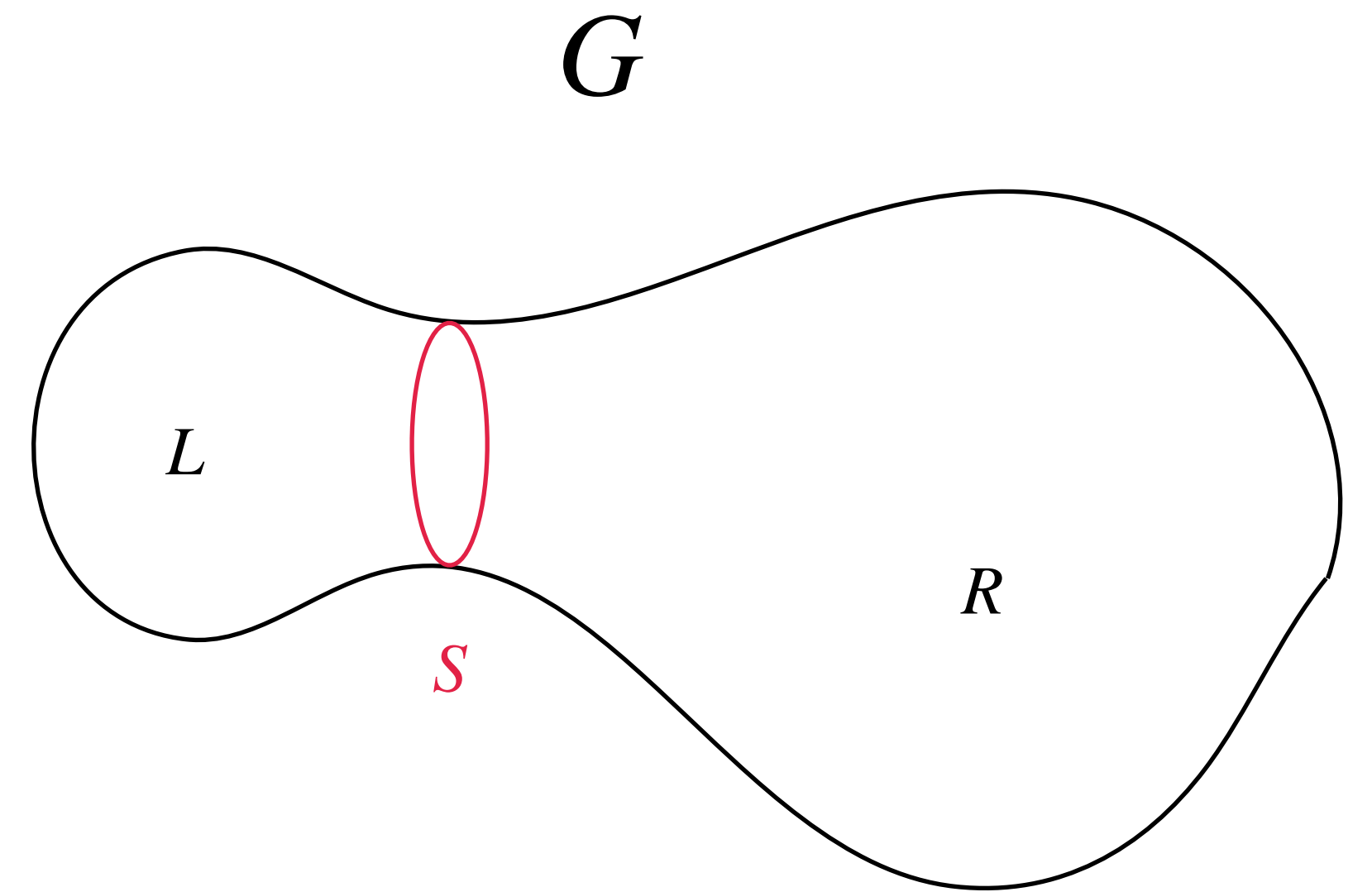
- Sequential algorithm idea.
- Obstacles in CONGEST implementation. $\left\{ \begin{array}{l} \text{Obstacle 1: when } |L| = o(\sqrt{n}), \text{ high congestion!} \\ \text{Obstacle 2: when } |L| = \omega(\sqrt{n}), \text{ high dilation!} \end{array} \right.$
- Solution to obstacle 1
- Solution to obstacle 2

Sequential Algorithm Idea [FNYSY'20]



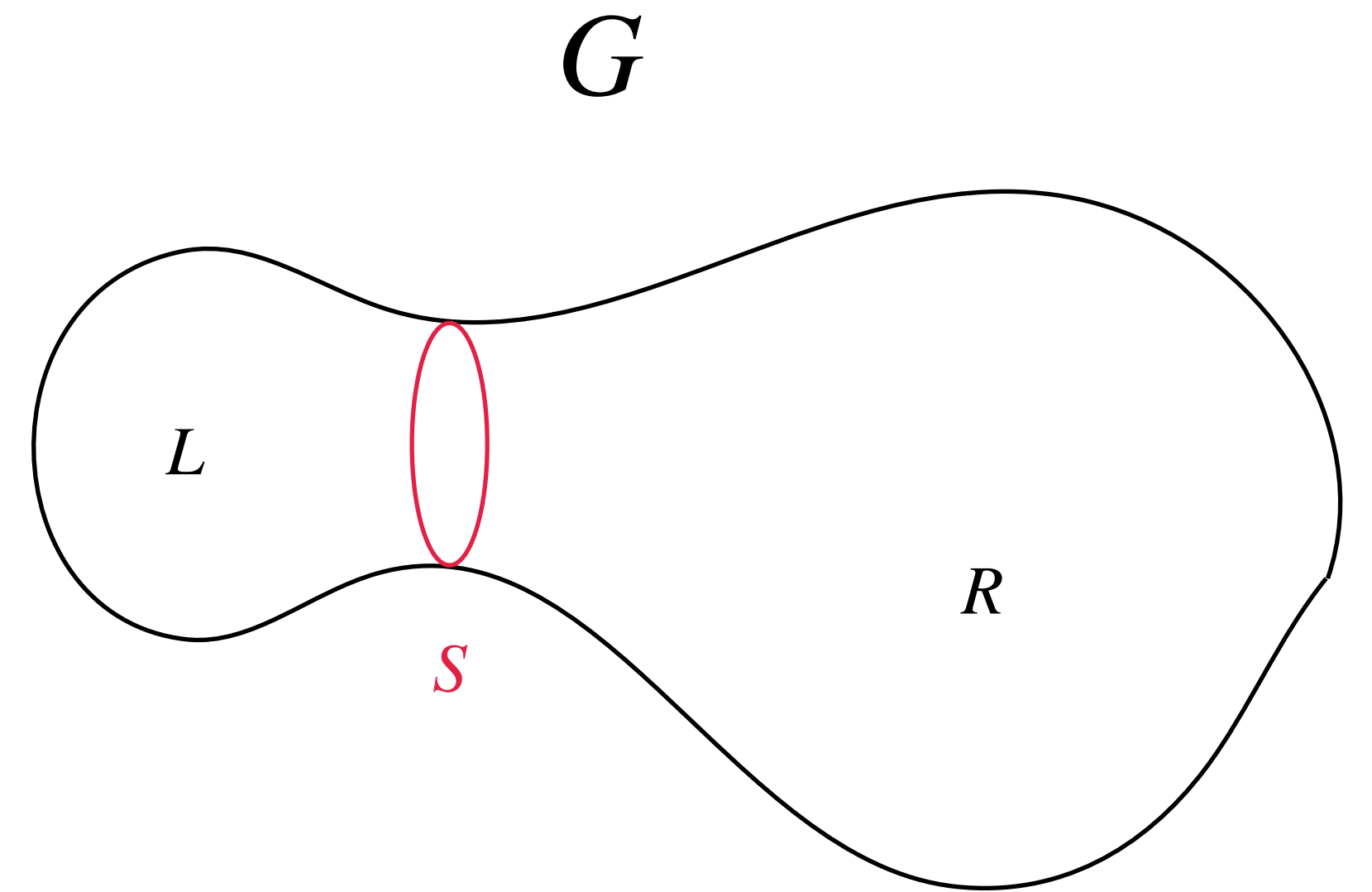
Sequential Algorithm Idea [FNYSY'20]

- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.



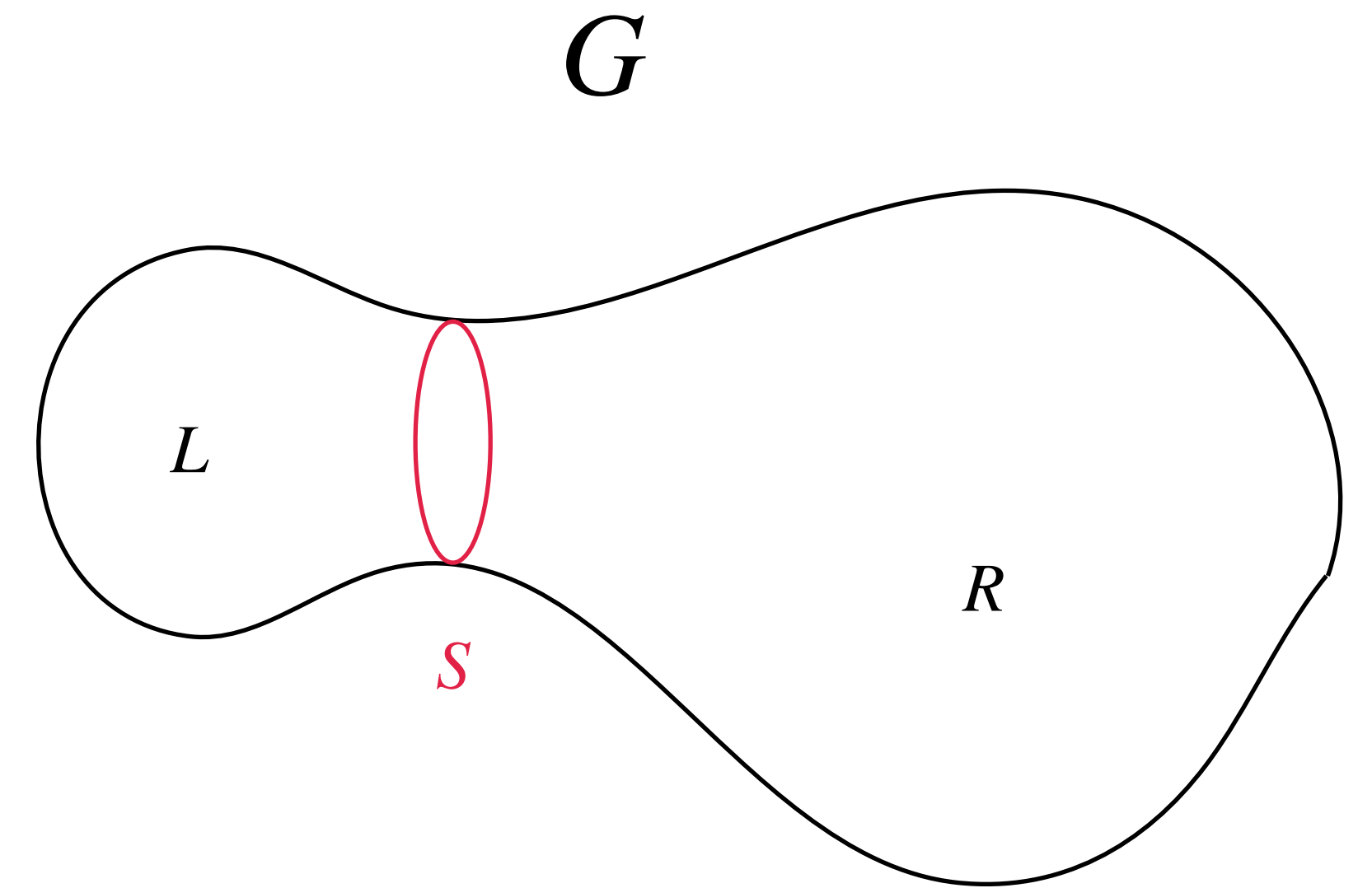
Sequential Algorithm Idea [FNYSY'20]

- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)



Sequential Algorithm Idea [FNYSY'20]

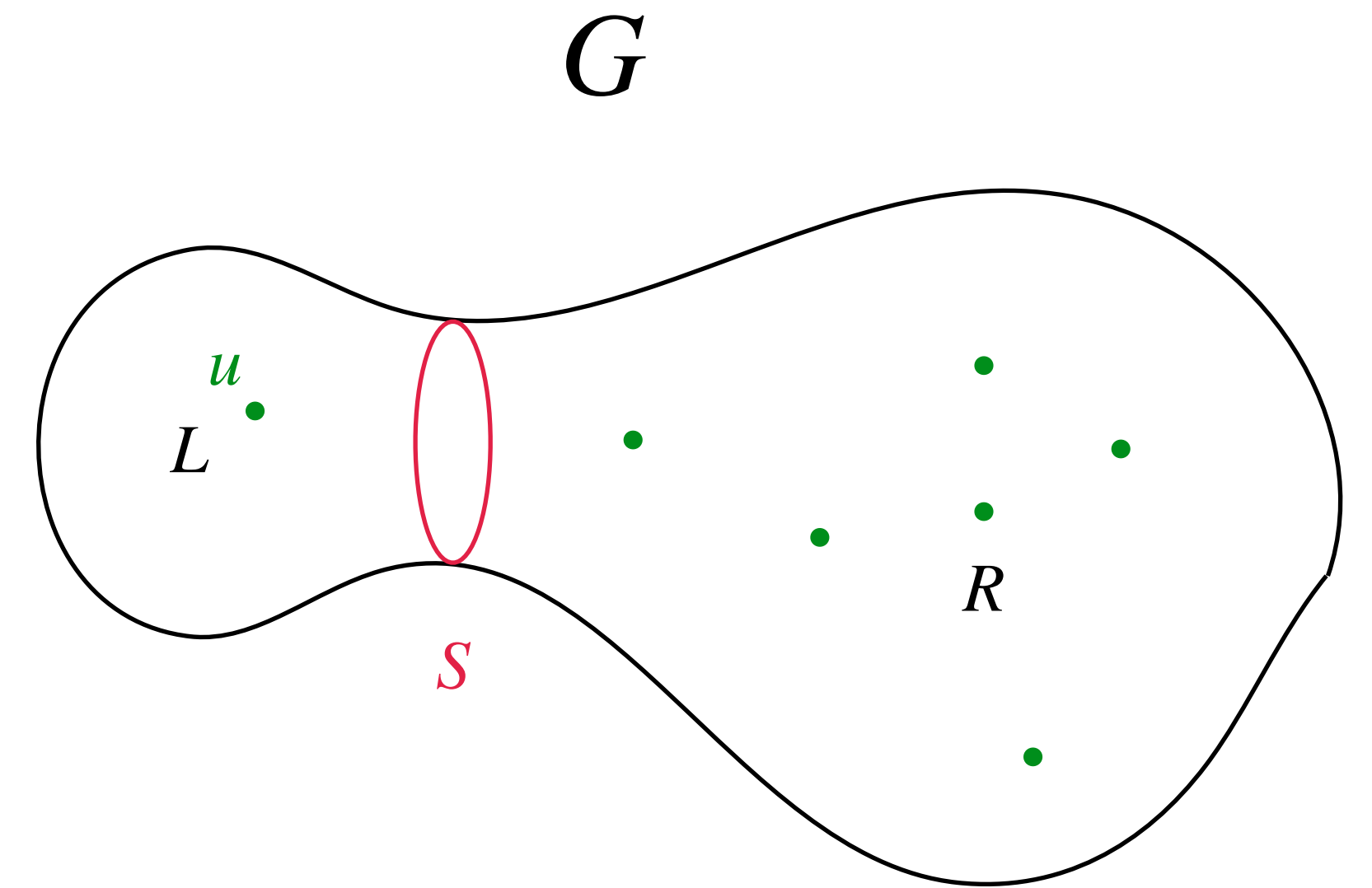
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

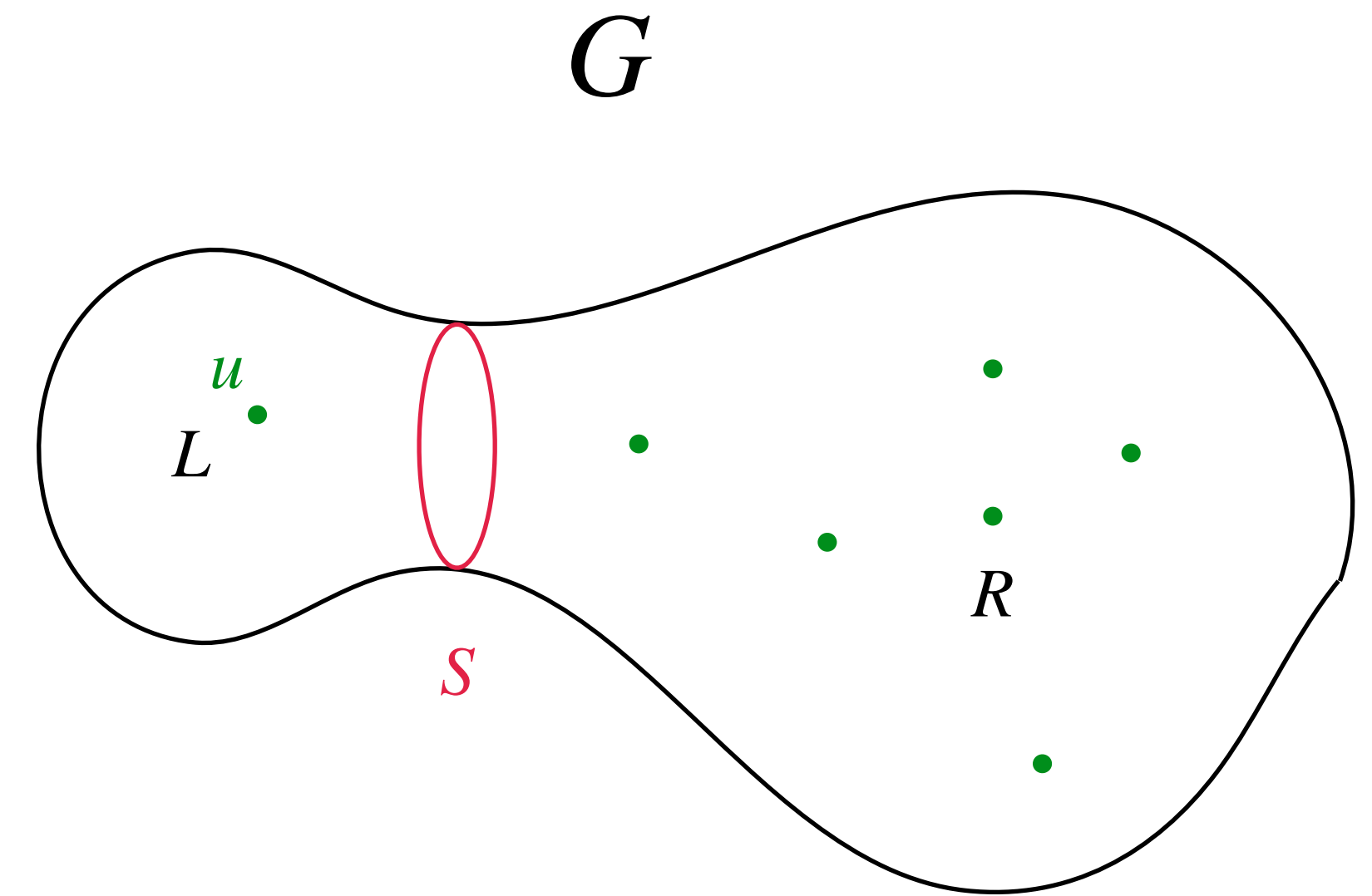
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

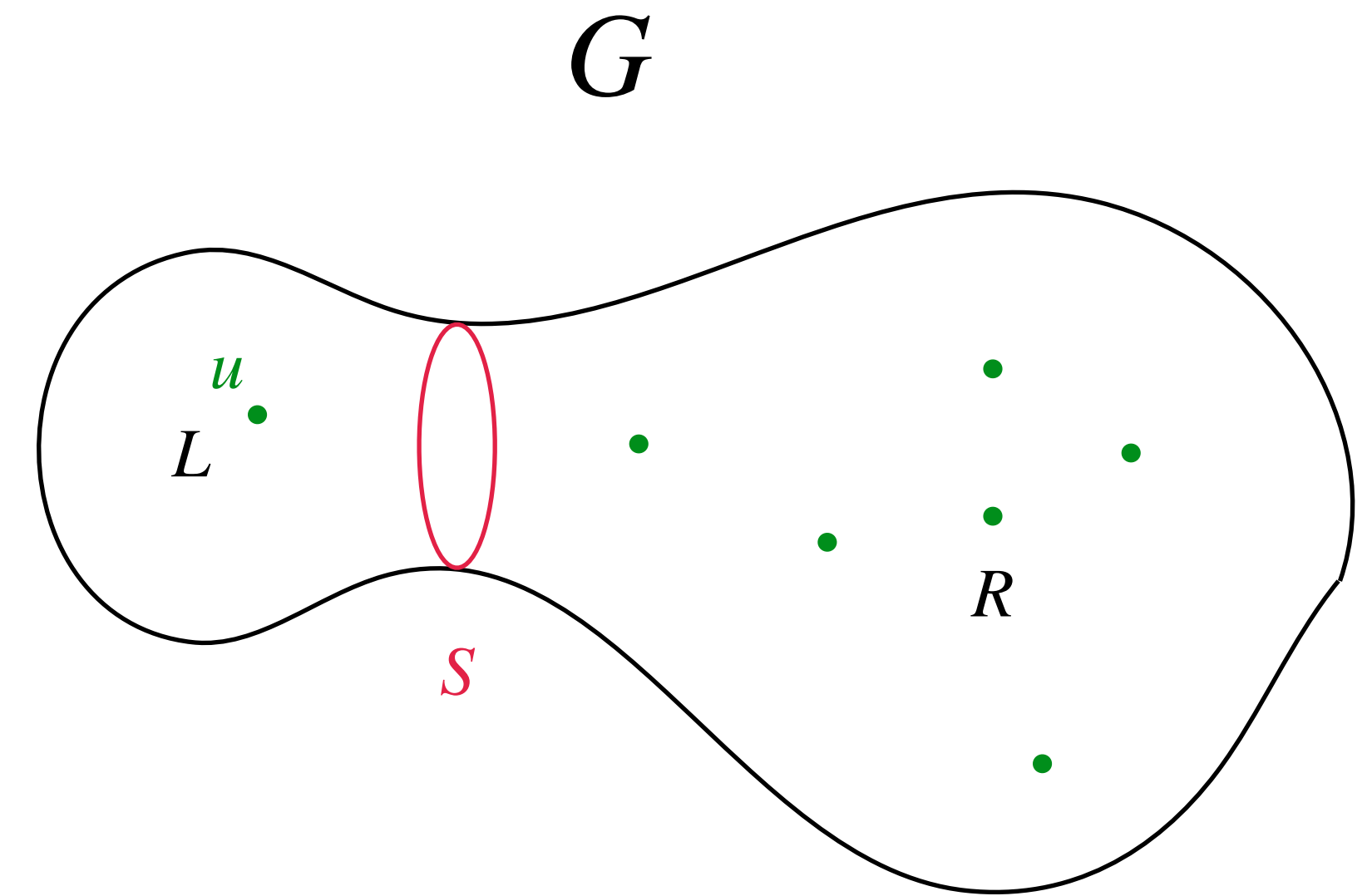
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

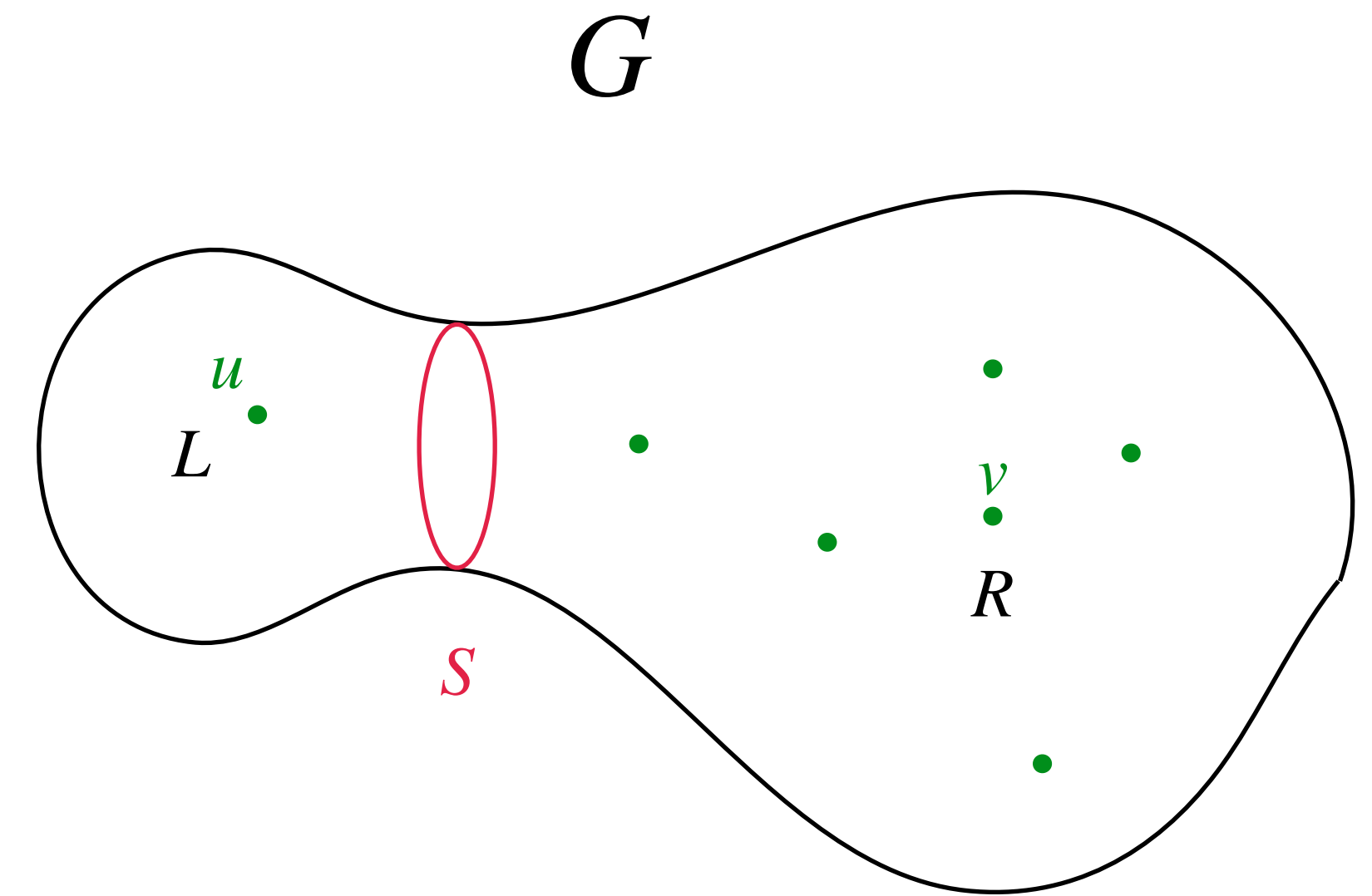
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

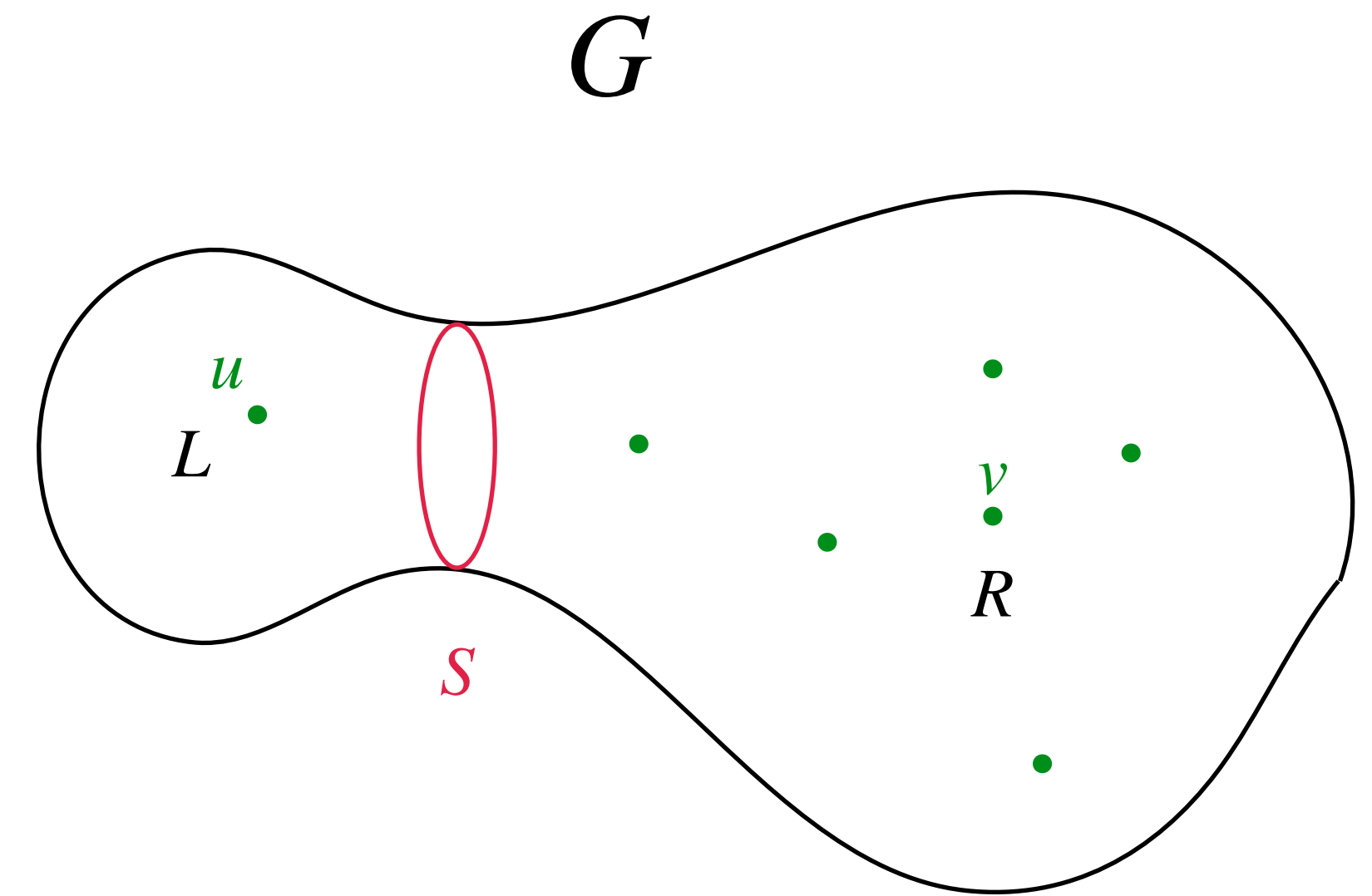
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

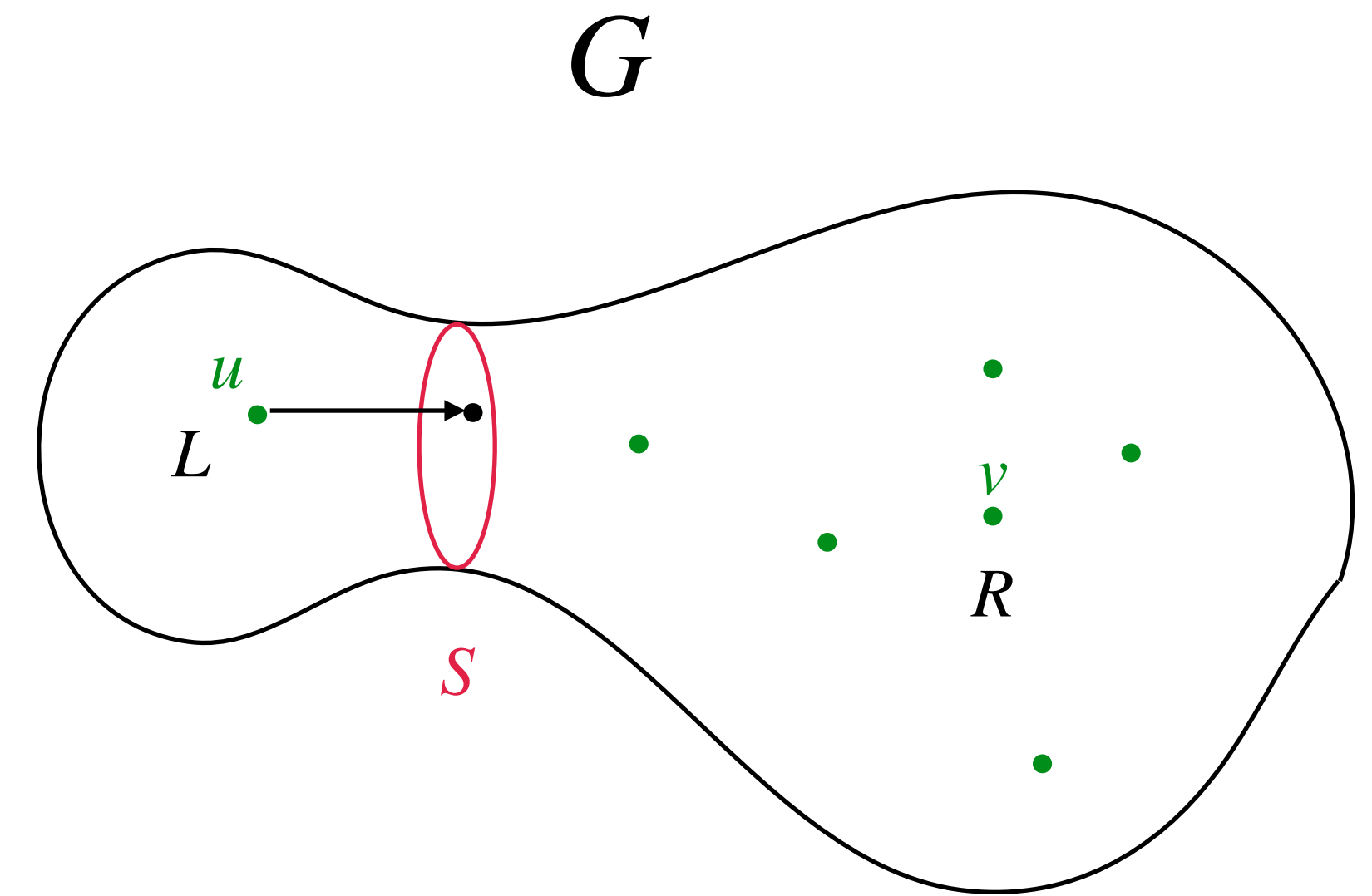
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

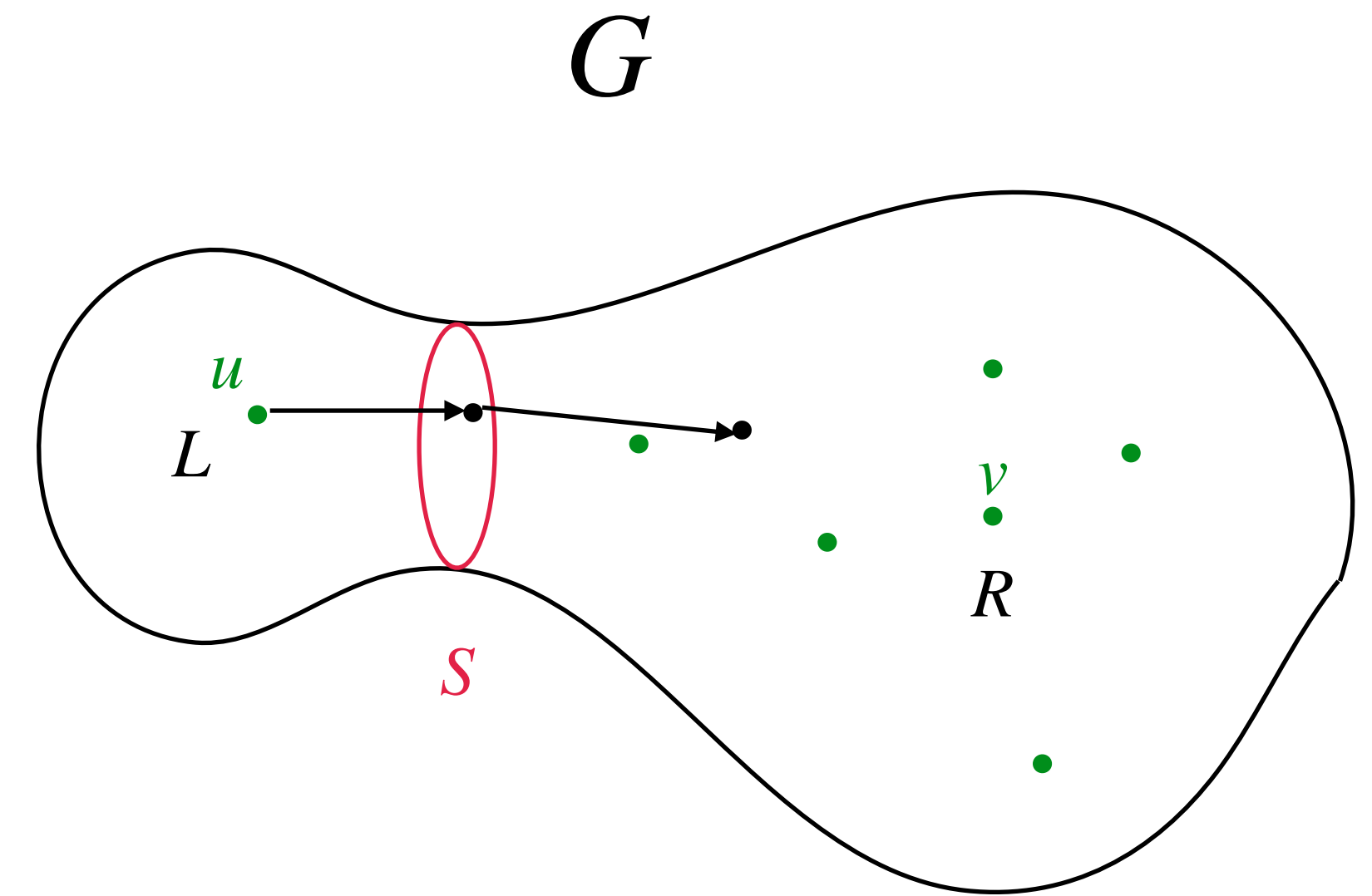
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

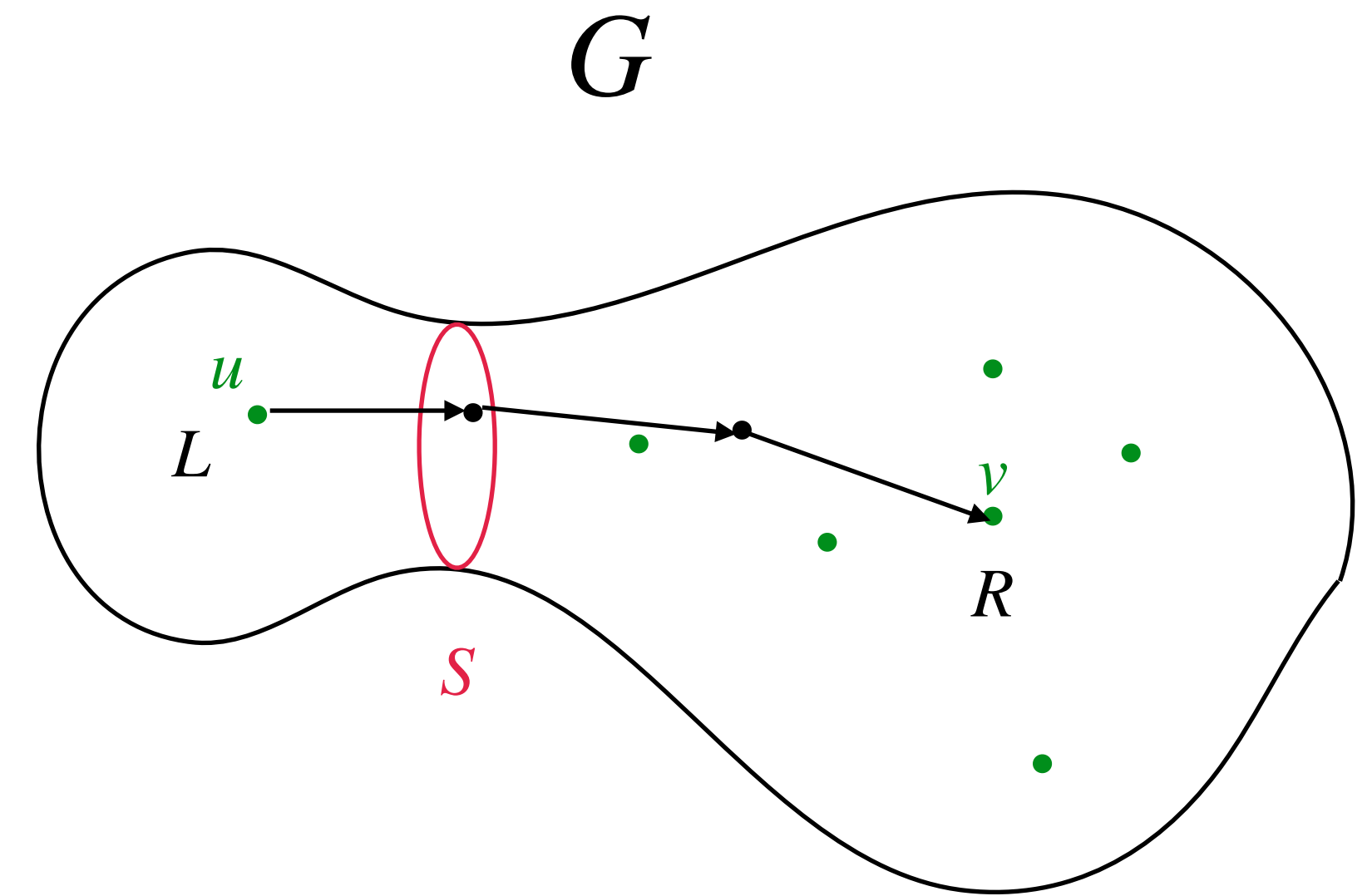
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

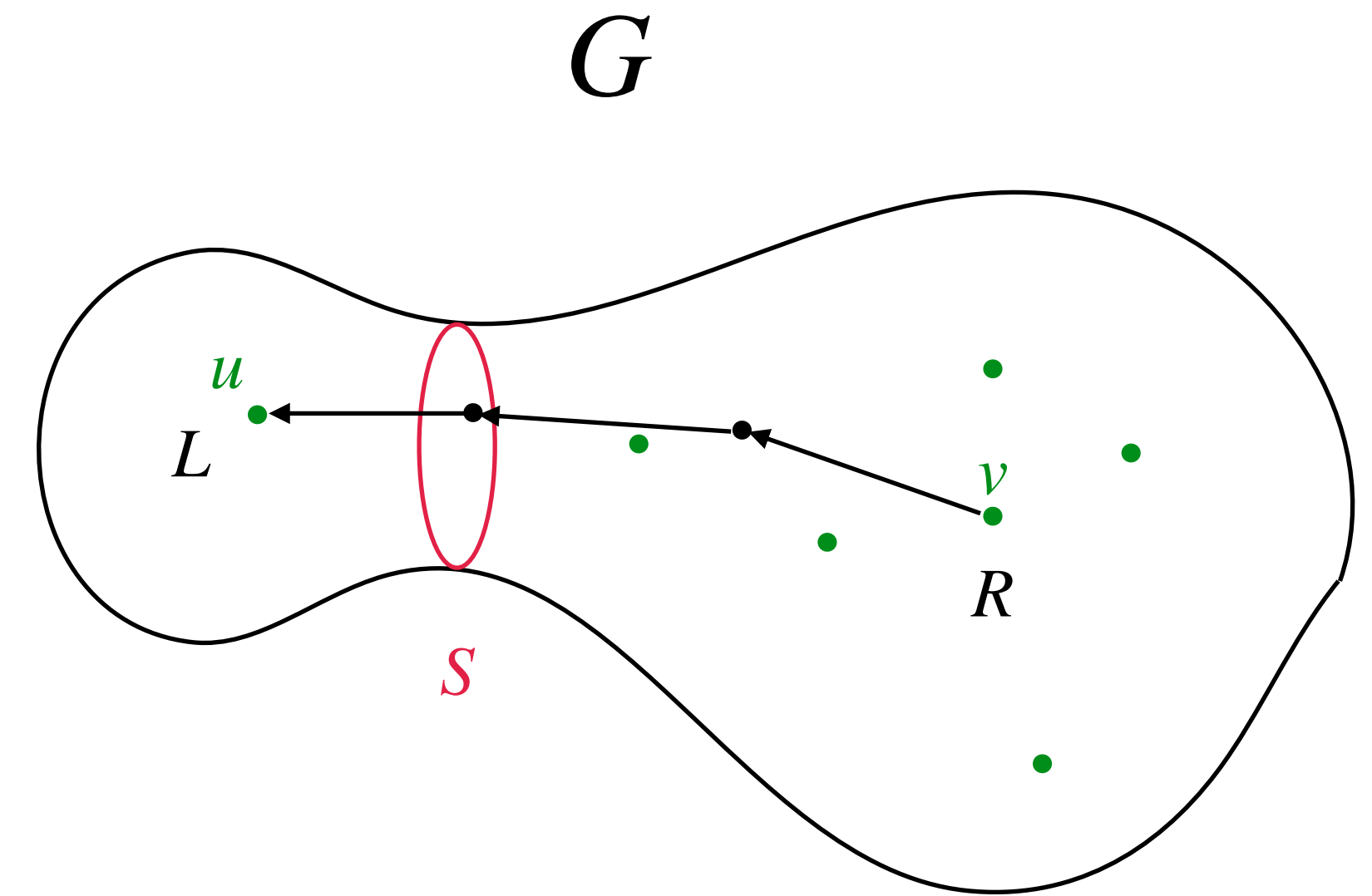
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

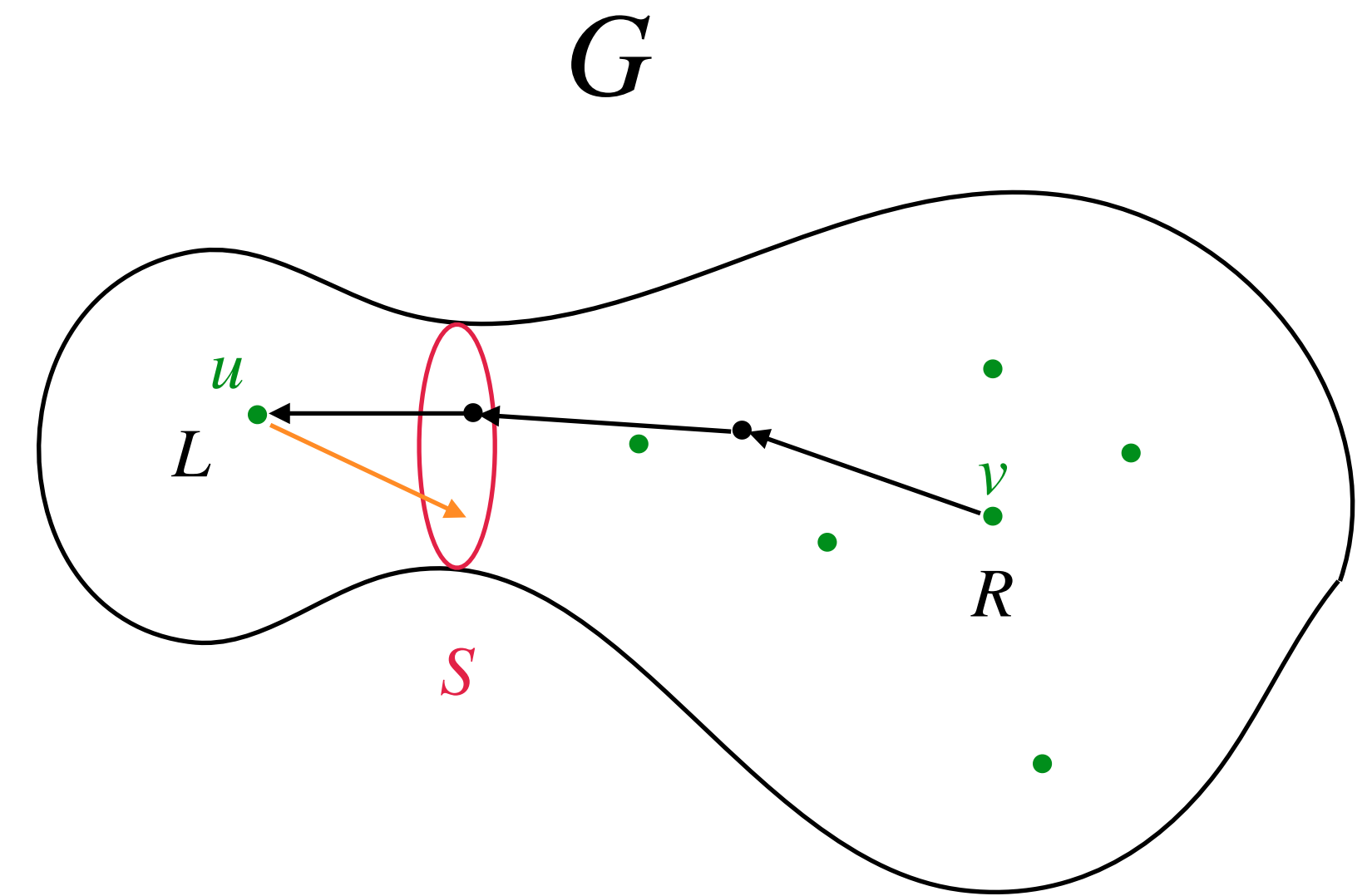
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

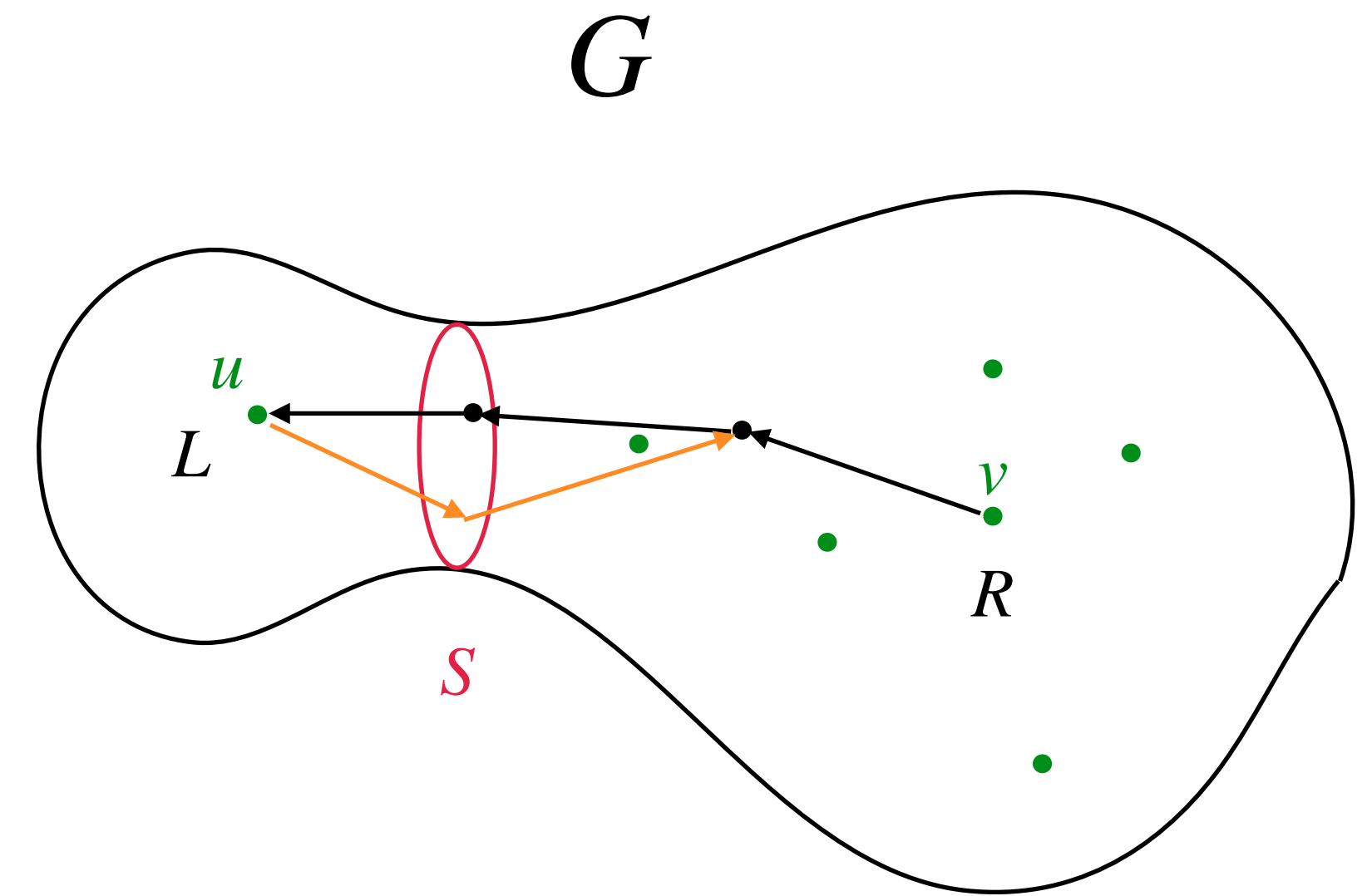
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

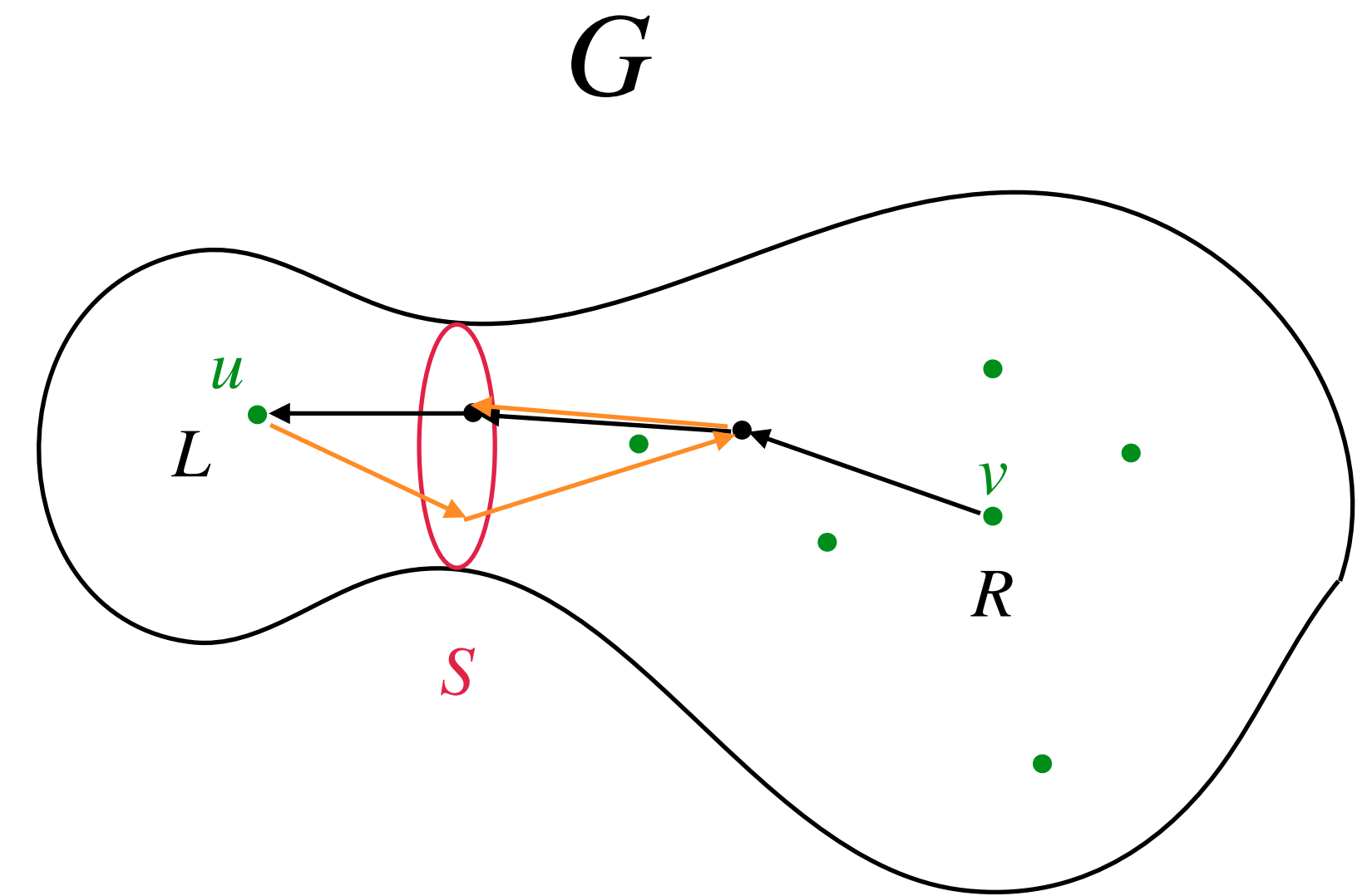
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

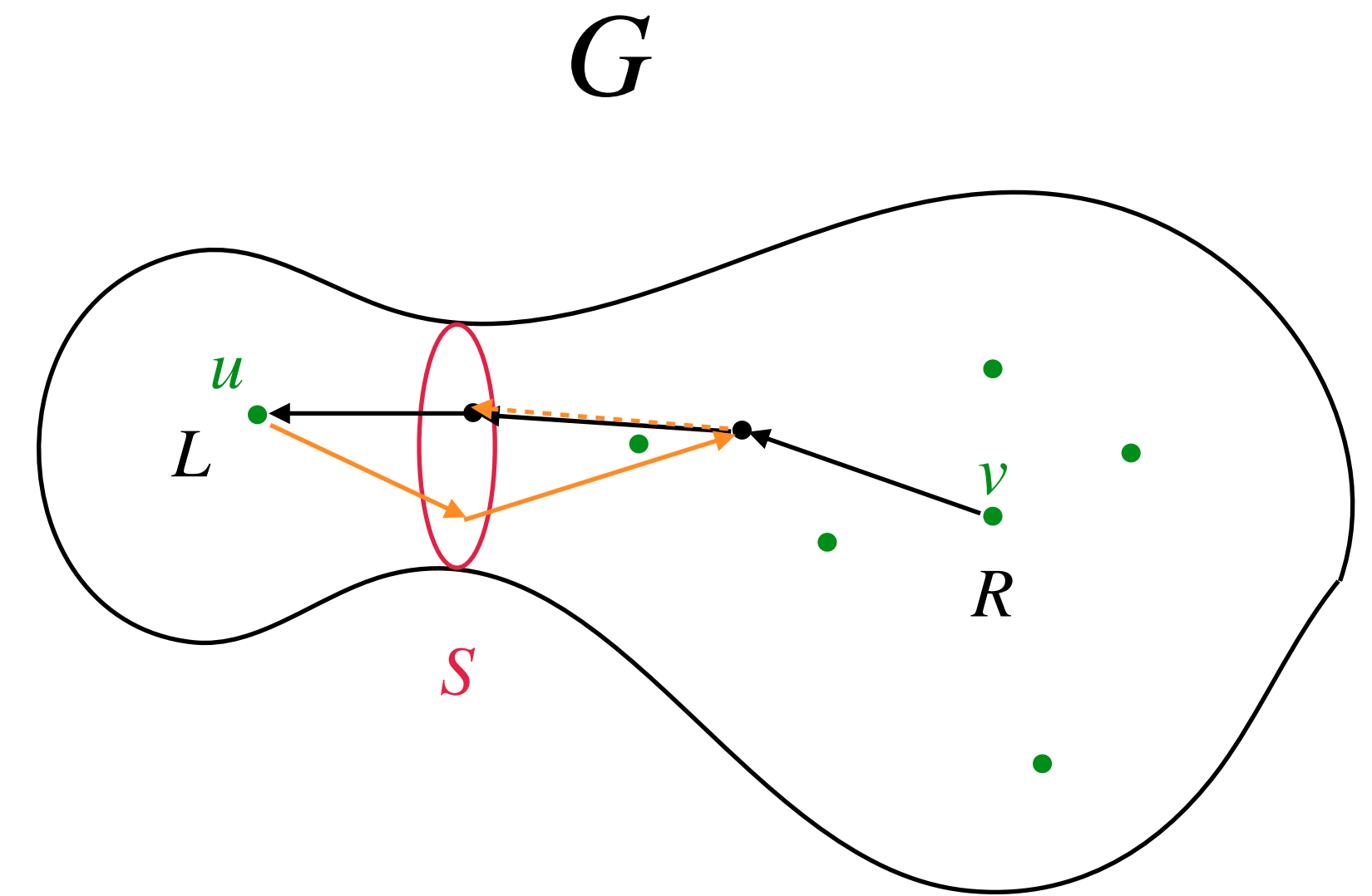
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

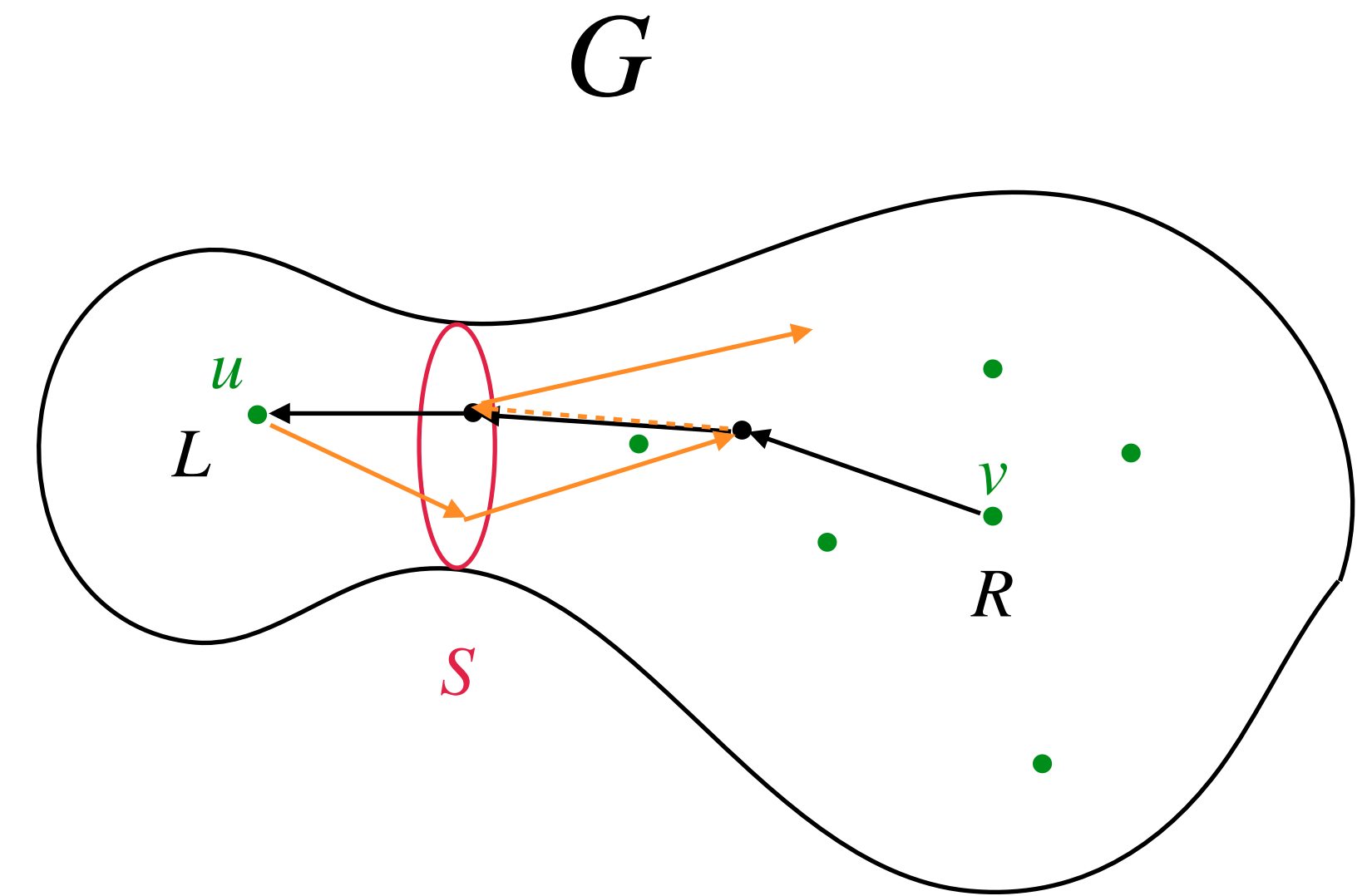
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

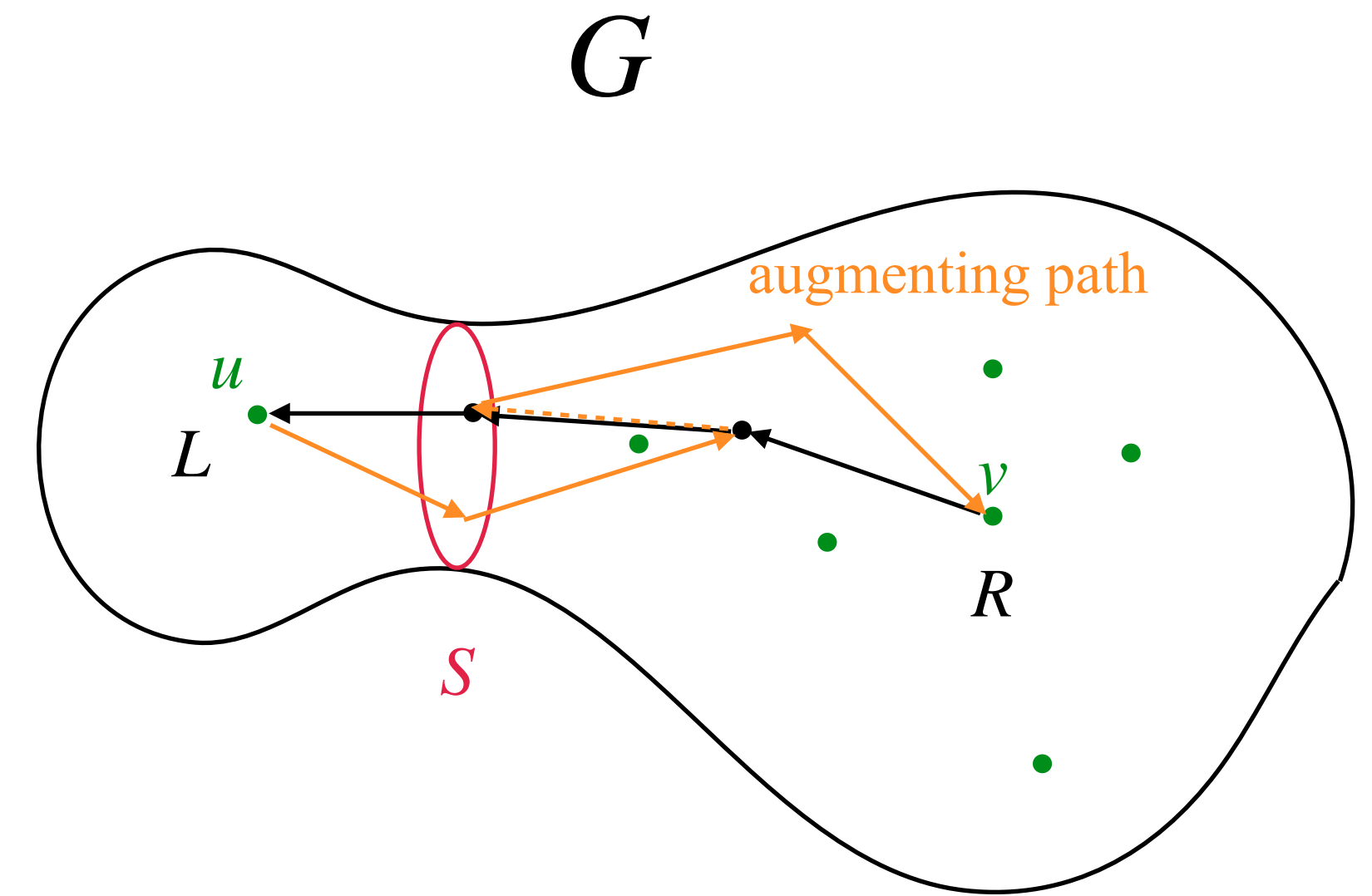
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

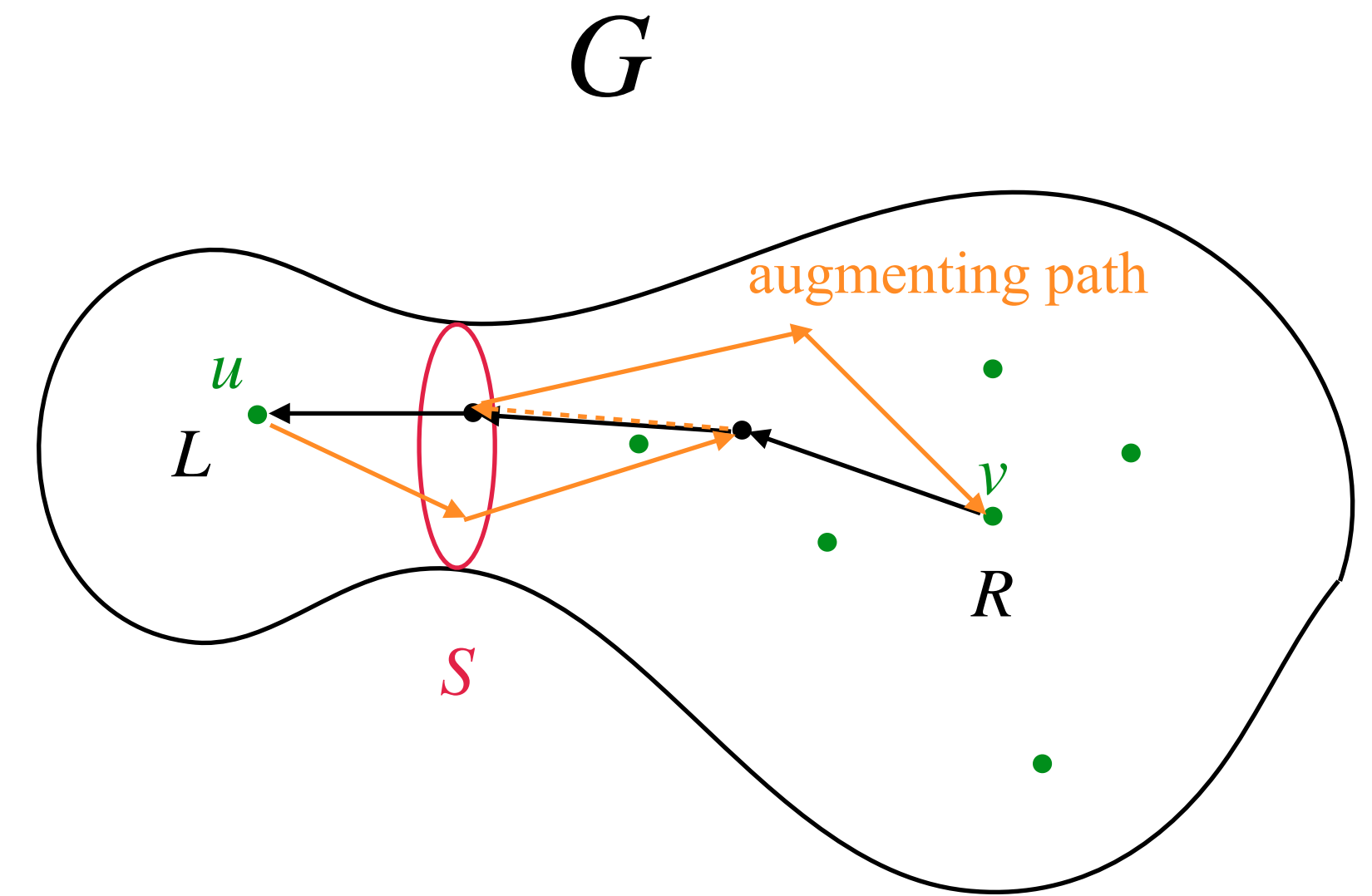
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
- Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
- For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

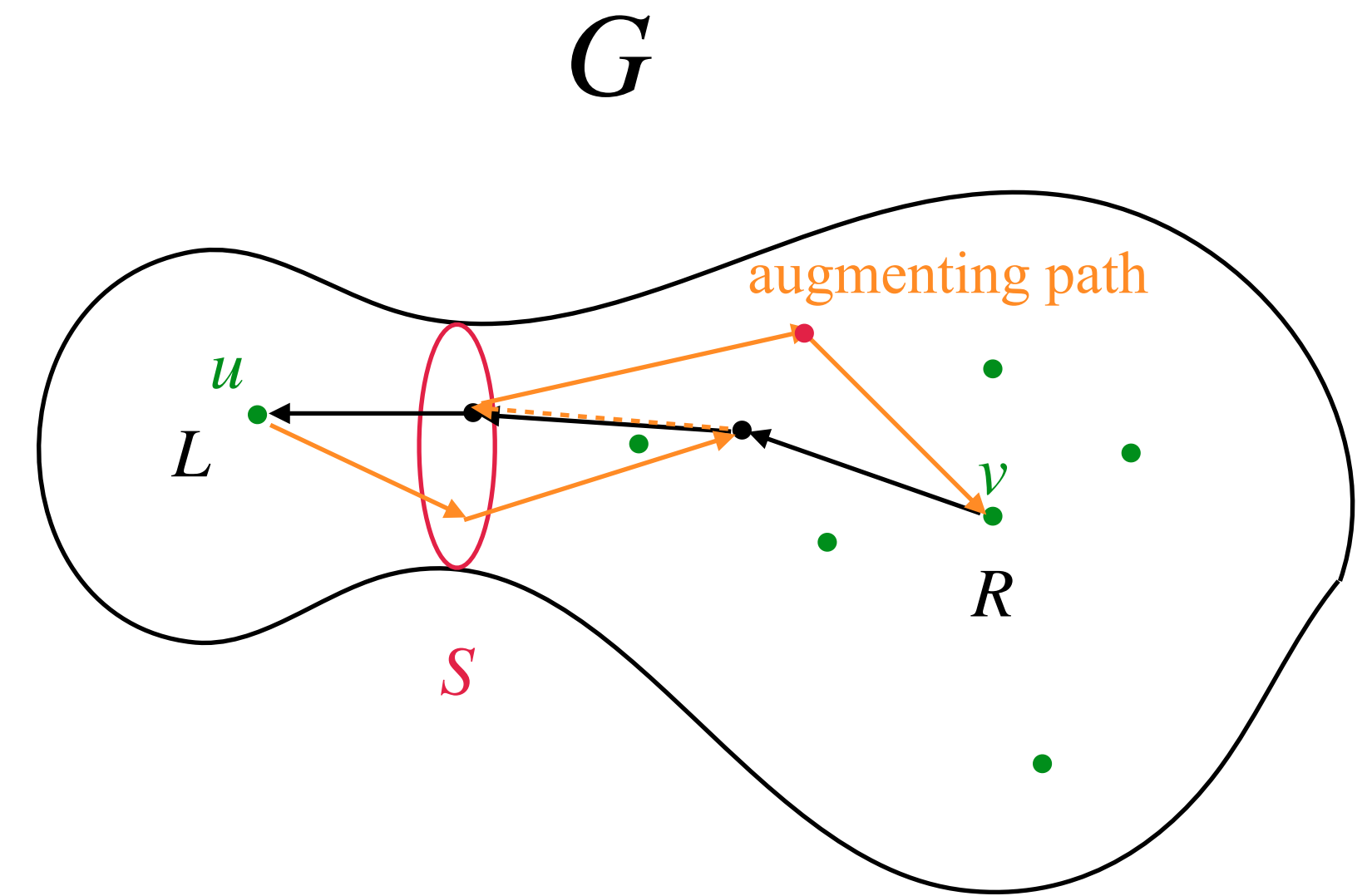
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
 - Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
 - For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .
- After DFS explores $\Omega(k|L|)$ vertices, pick a **random vertex** as the termination of **augmenting path**.



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

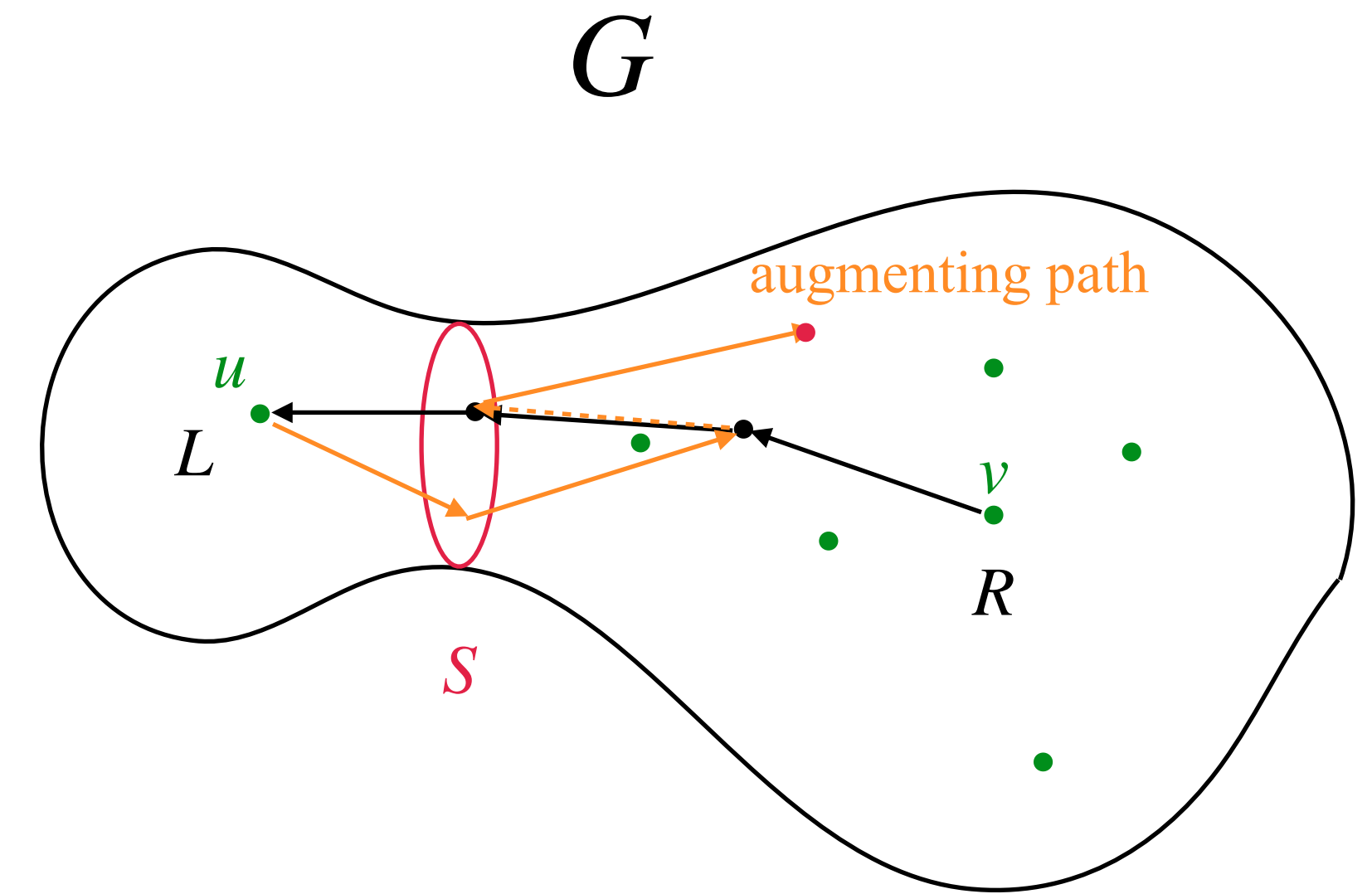
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
 - Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
 - For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .
- After DFS explores $\Omega(k|L|)$ vertices, pick a **random vertex** as the termination of **augmenting path**.



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

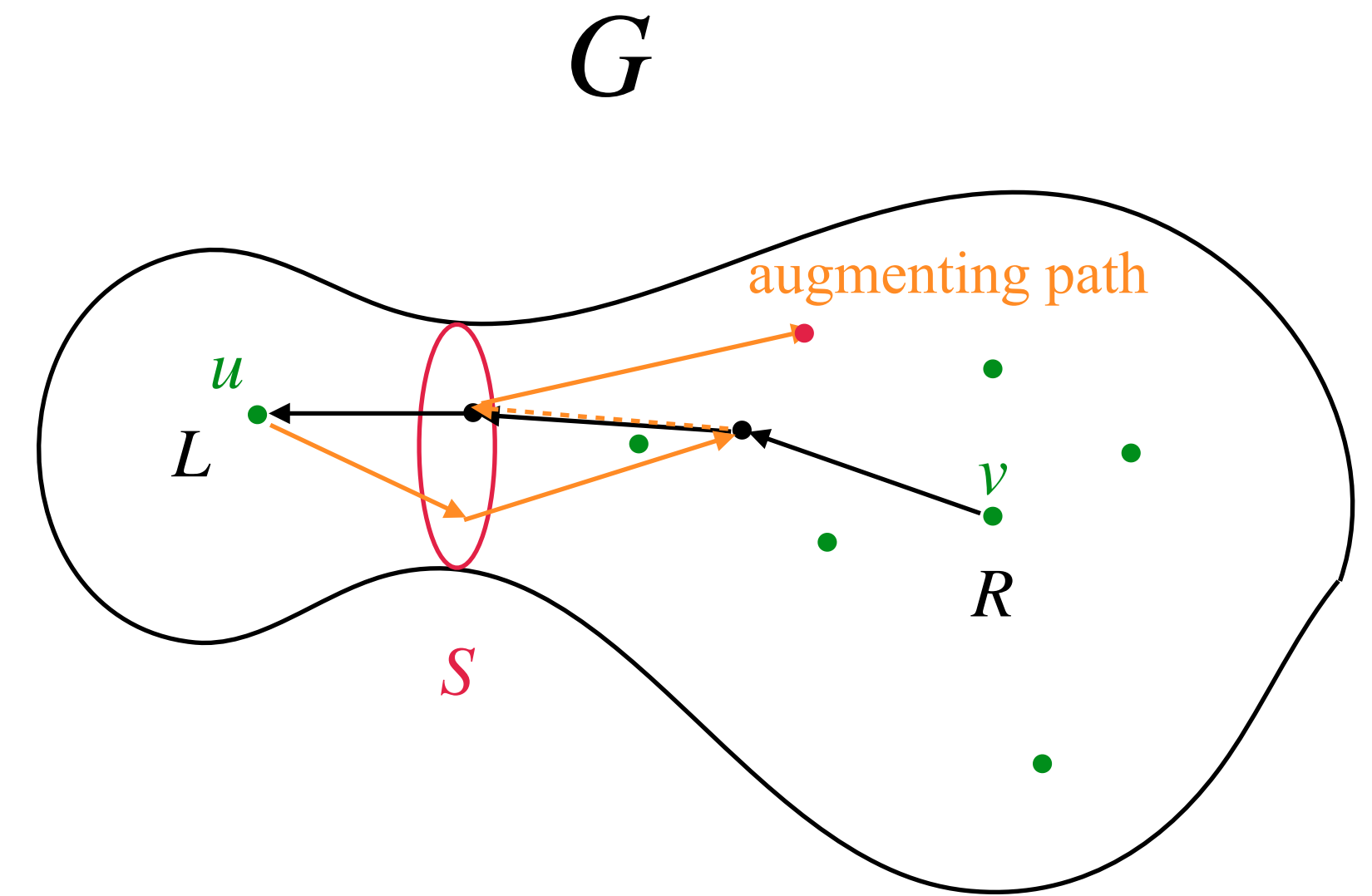
- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
 - Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
 - For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .
- After DFS explores $\Omega(k|L|)$ vertices, pick a **random vertex** as the termination of **augmenting path**.



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

Sequential Algorithm Idea [FNYSY'20]

- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
 - Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
 - For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .
- After DFS explores $\Omega(k|L|)$ vertices, pick a **random vertex** as the termination of **augmenting path**.

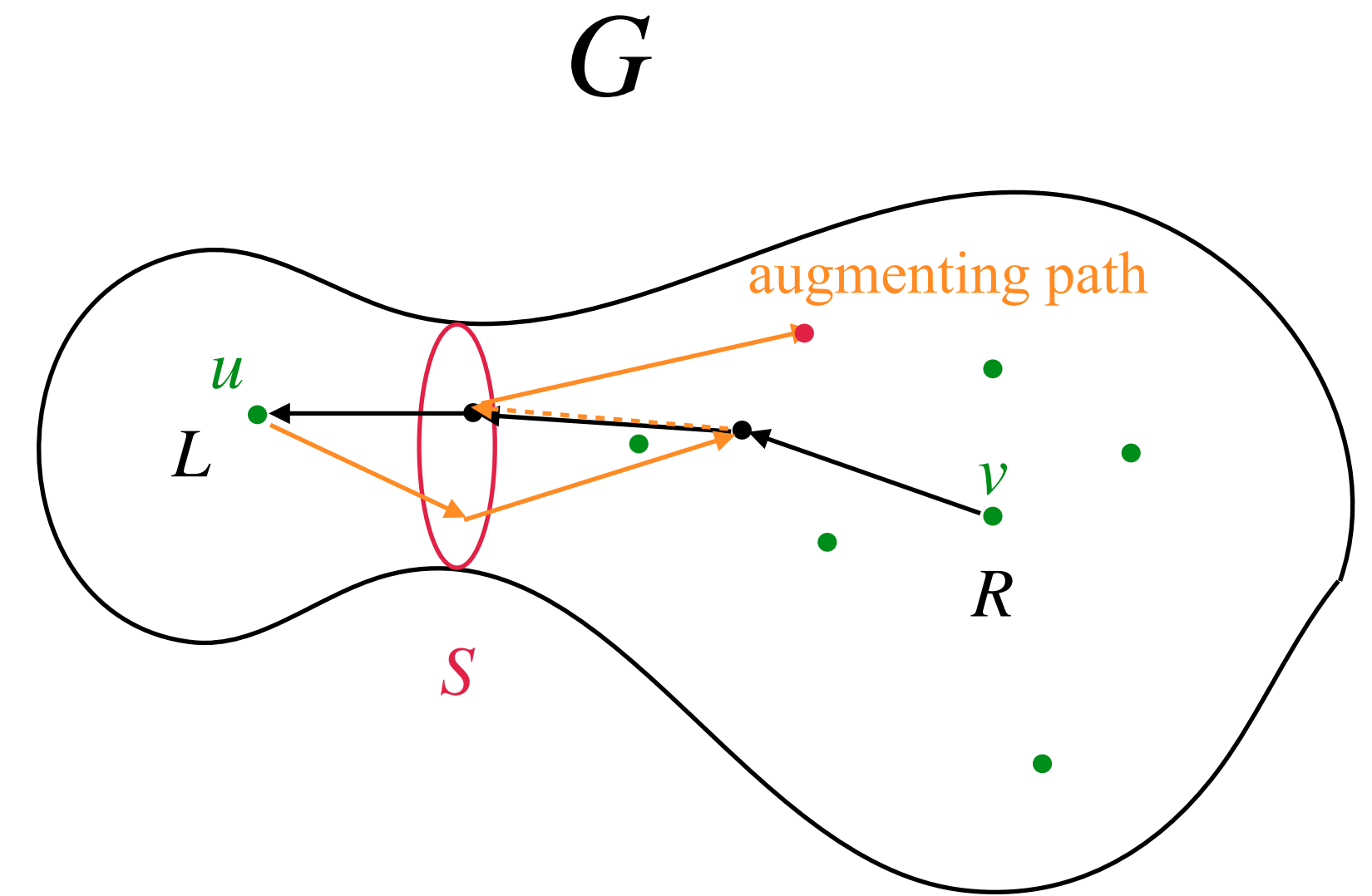


w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

All augmenting paths end in R w.p. $\geq (1 - 1/k)^k$

Sequential Algorithm Idea [FNYSY'20]

- Assumptions:
 - $|L|$ is known.
 - $|S| = O(1)$.
 - Sample each node into R with probability $1/|L|$ ($|R| \approx n/|L|$)
 - For each $u \in R$ do
 - find minimum vertex cut to another $v \in R$
by running k times of DFS to find **augmenting paths** to v .
- After DFS explores $\Omega(k|L|)$ vertices, pick a **random vertex** as the termination of **augmenting path**.



w.p. ≥ 0.1 , $|L \cap R| = 1$ and $S \cap R = \emptyset$

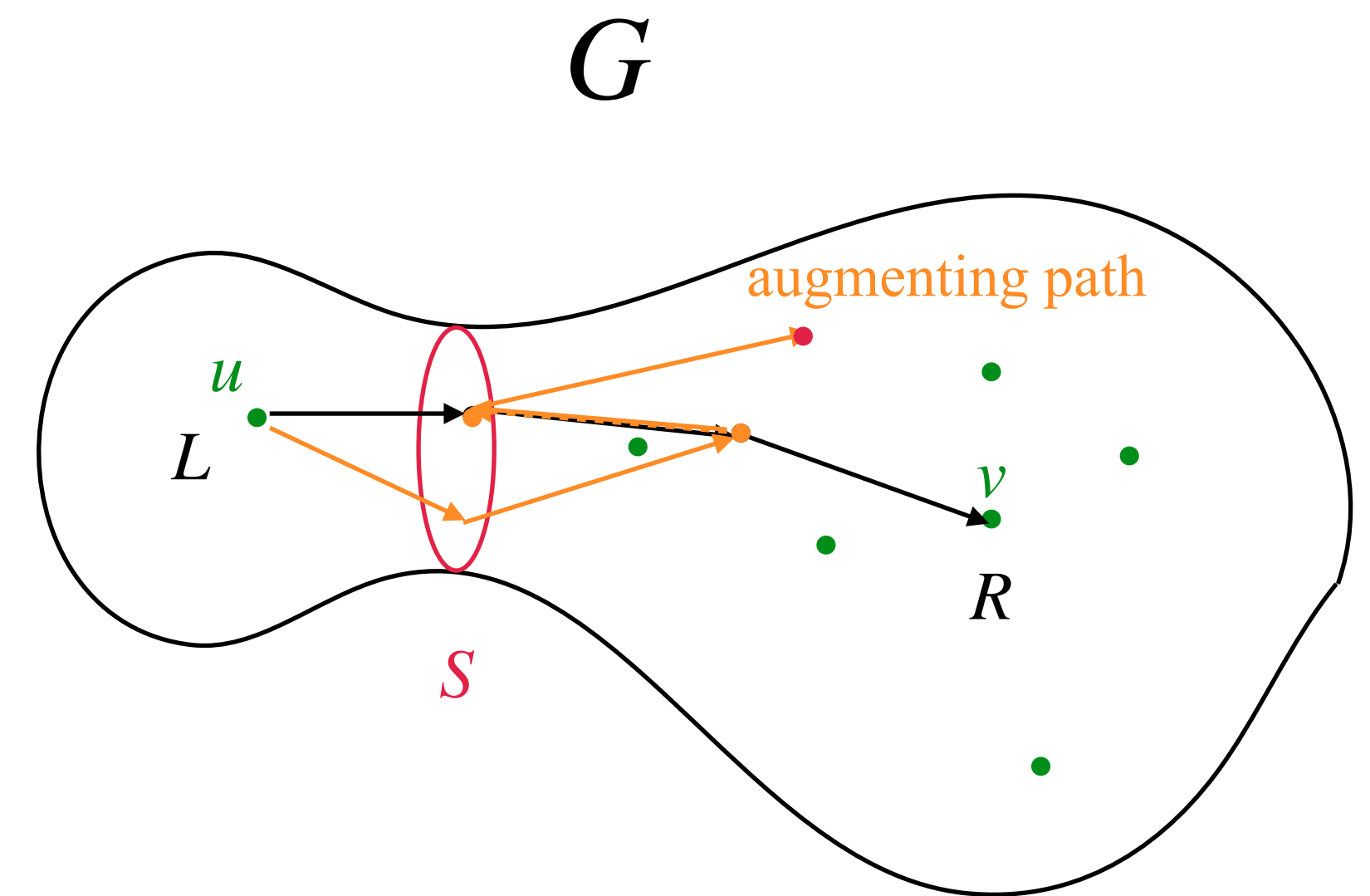
All augmenting paths end in R w.p. $\geq (1 - 1/k)^k$

One sided error!

Outline

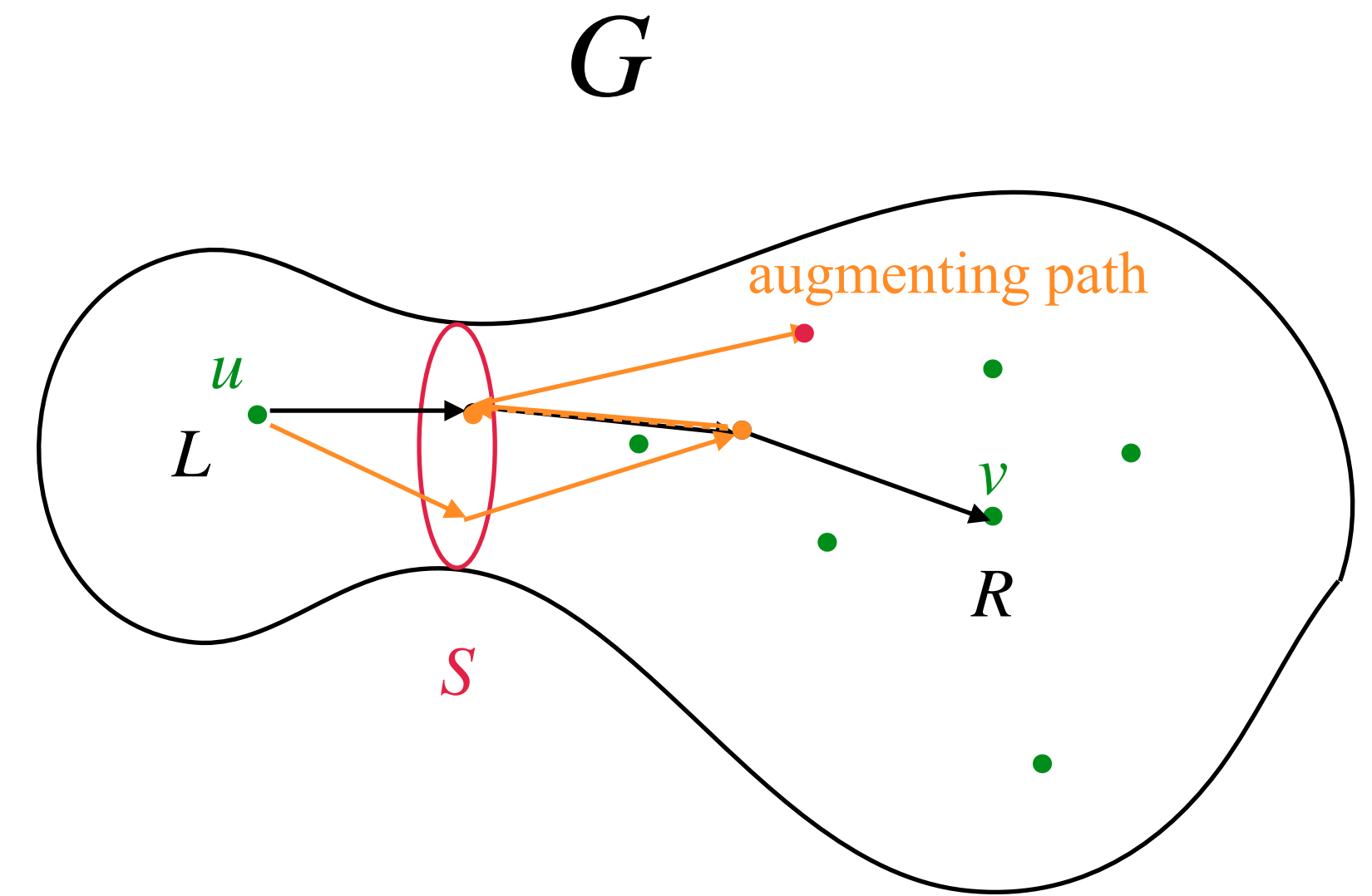
- Sequential algorithm idea.
- **Obstacles in CONGEST implementation.** $\left\{ \begin{array}{l} \text{Obstacle 1: when } |L| = o(\sqrt{n}), \text{ high congestion!} \\ \text{Obstacle 2: when } |L| = \omega(\sqrt{n}), \text{ high dilation!} \end{array} \right.$
- Solution to obstacle 1
- Solution to obstacle 2

CONGEST implementation



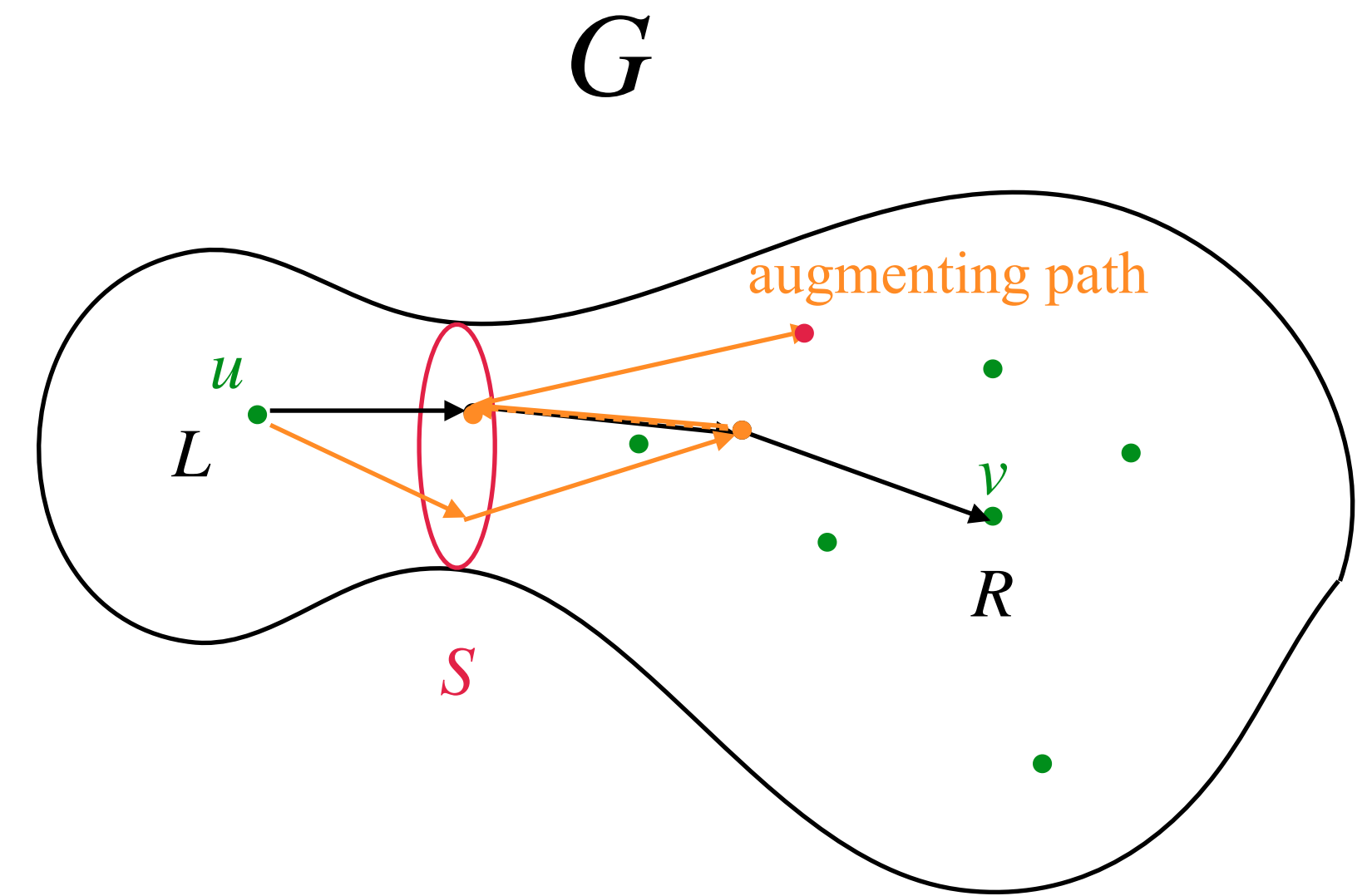
CONGEST implementation

- From $n/|L|$ vertices in R , run k times of DFS and terminate upon



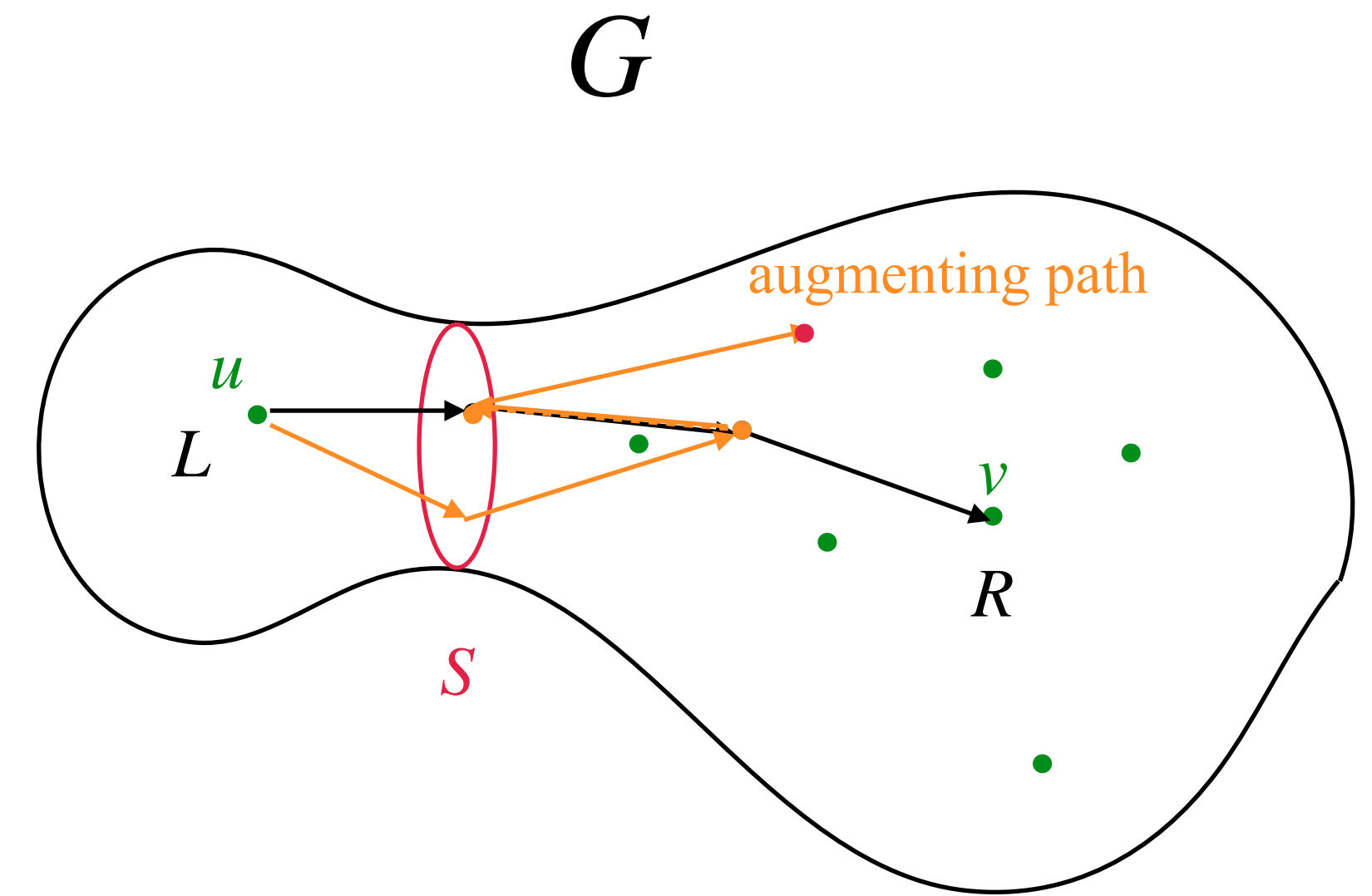
CONGEST implementation

- From $n/|L|$ vertices in R , run k times of DFS and **terminate** upon
 - reaching another $v \in R$, or
 - having explored $\Omega(k|L|)$ vertices.



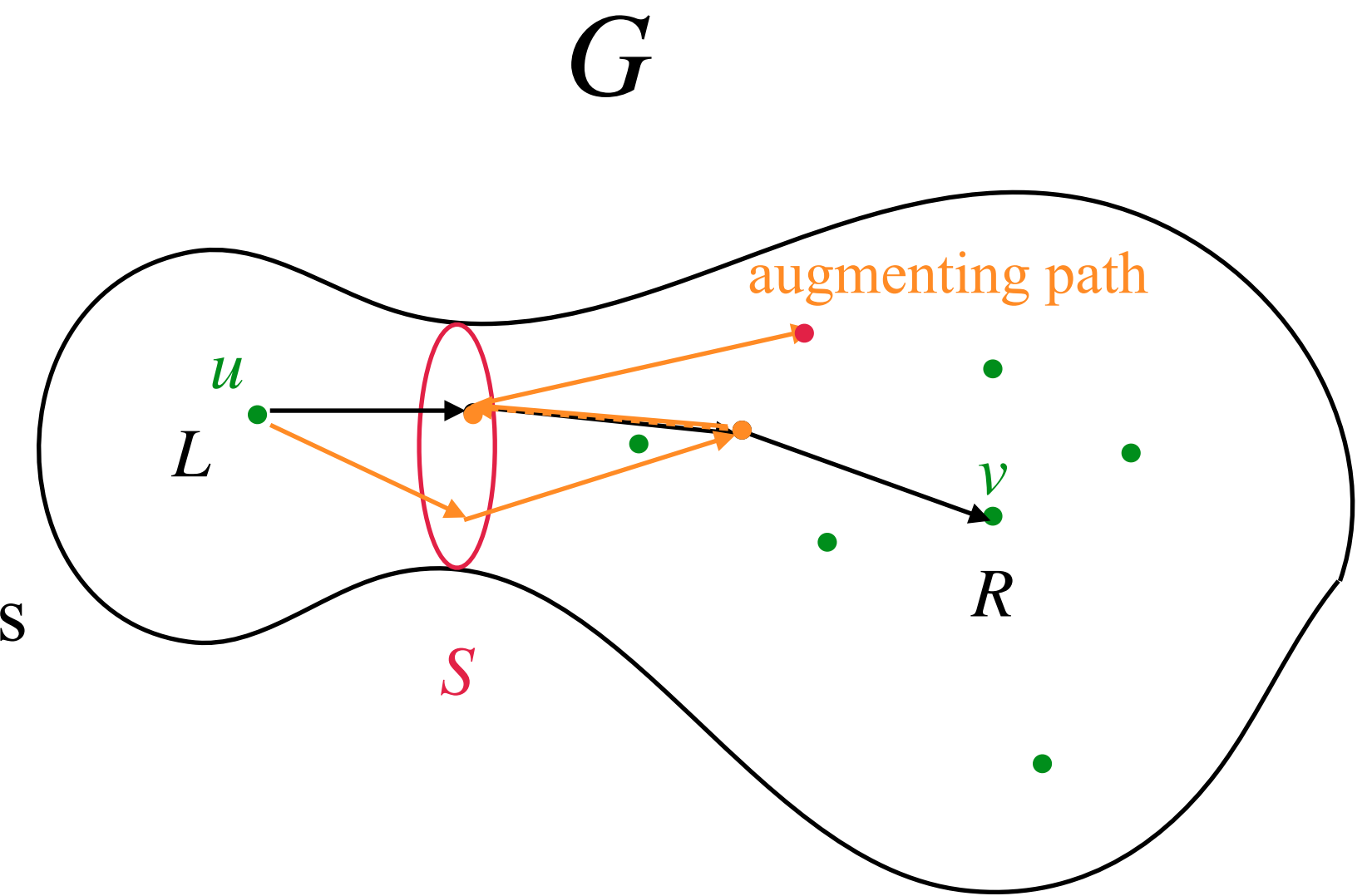
CONGEST implementation

- From $n/|L|$ vertices in R , run k times of DFS and **terminate** upon
 - reaching another $v \in R$, or
 - having explored $\Omega(k|L|)$ vertices.
- [Ghaffari'15] simulate $n/|L|$ algorithms in $\tilde{O}(\text{congestion} + \text{dilation})$



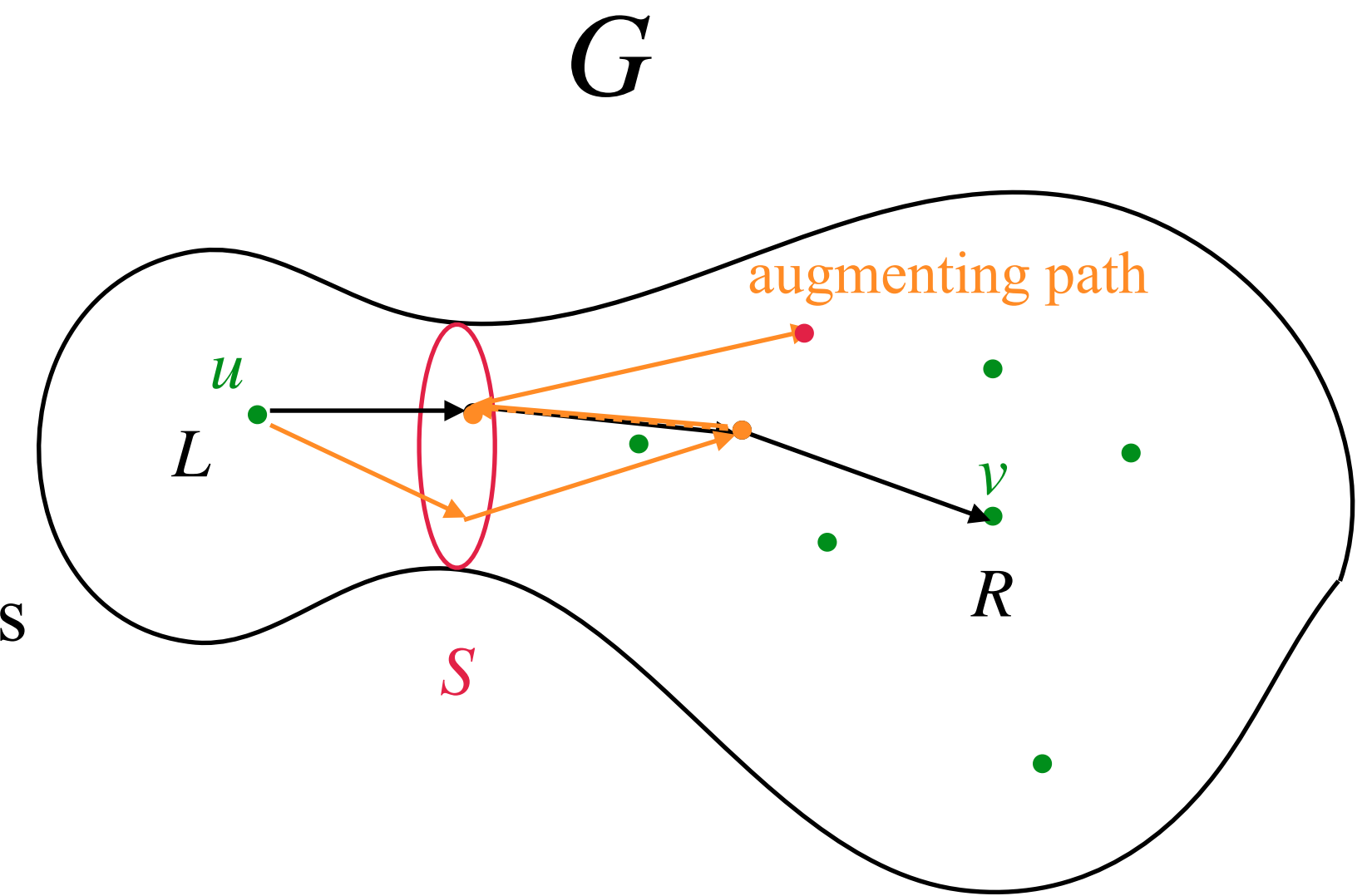
CONGEST implementation

- From $n/|L|$ vertices in R , run k times of DFS and **terminate** upon
 - reaching another $v \in R$, or
 - having explored $\Omega(k|L|)$ vertices.
- [Ghaffari'15] simulate $n/|L|$ algorithms in $\tilde{O}(\text{congestion} + \text{dilation})$
 - **Congestion**: maximum congestion of each edges; the congestion of an edge is the number of messages sent by all algorithm through this edge $n/|L|$
 - **Dilation**: maximum round complexity among all algorithms $O(k|L|)$



CONGEST implementation

- From $n/|L|$ vertices in R , run k times of DFS and **terminate** upon
 - reaching another $v \in R$, or
 - having explored $\Omega(k|L|)$ vertices.
- [Ghaffari'15] simulate $n/|L|$ algorithms in $\tilde{O}(\text{congestion} + \text{dilation})$
 - **Congestion**: maximum congestion of each edges; the congestion of an edge is the number of messages sent by all algorithm through this edge $n/|L|$
 - **Dilation**: maximum round complexity among all algorithms $O(k|L|)$



Obstacle 1: when $|L| = o(\sqrt{n})$, high **congestion**!

Obstacle 2: when $|L| = \omega(\sqrt{n})$, high **dilation**!

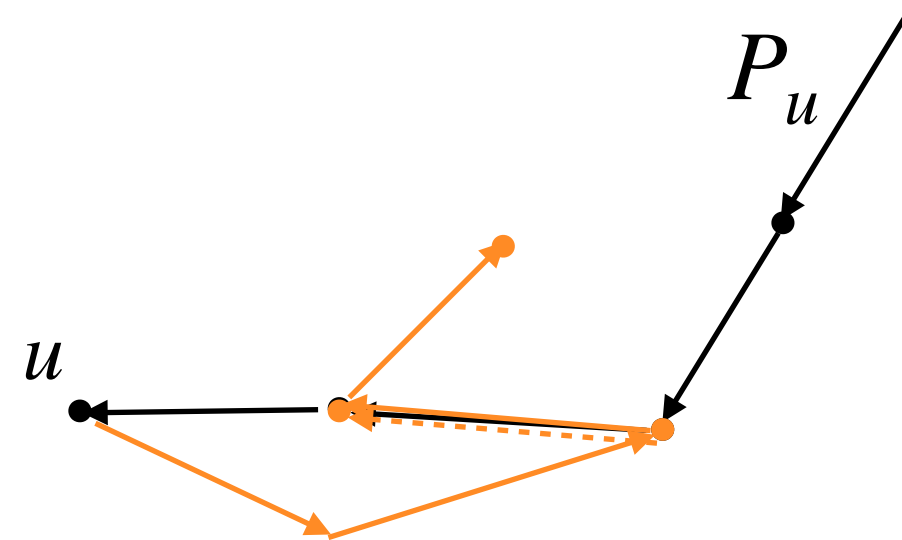
Outline

- Sequential algorithm idea.
- Obstacles in CONGEST implementation. $\left\{ \begin{array}{l} \text{Obstacle 1: when } |L| = o(\sqrt{n}), \text{ high congestion!} \\ \text{Obstacle 2: when } |L| = \omega(\sqrt{n}), \text{ high dilation!} \end{array} \right.$
- **Solution to obstacle 1**
- Solution to obstacle 2

when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

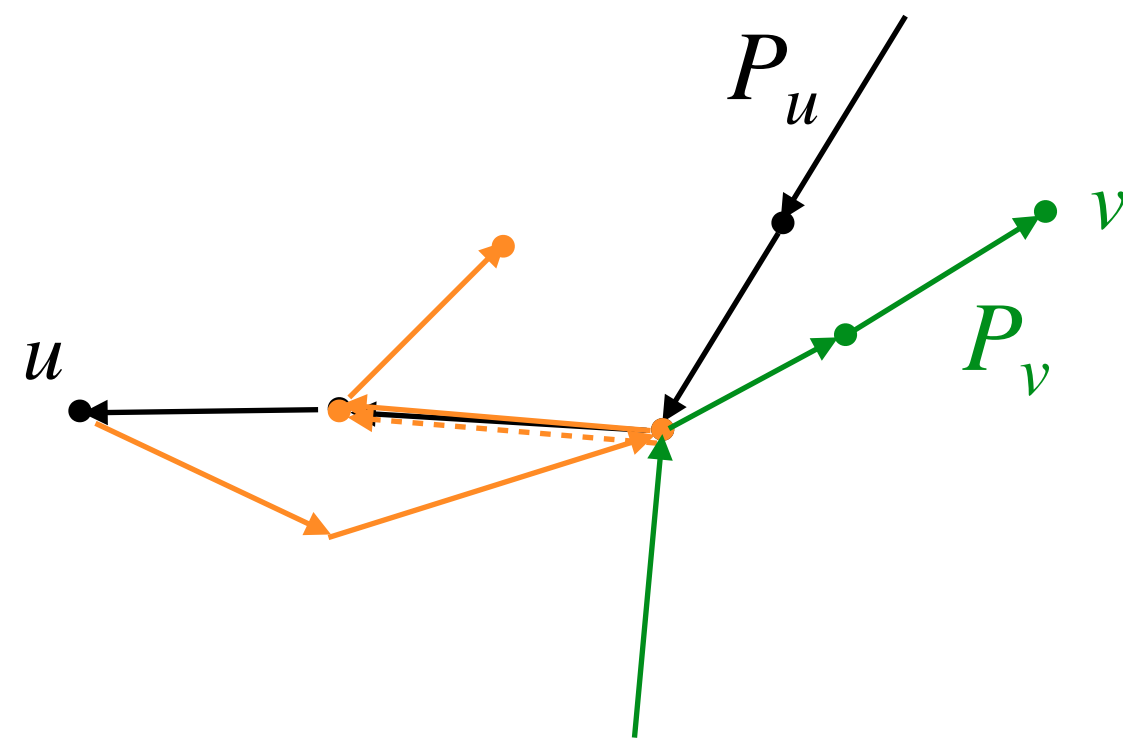
Depends on previous explored edges P_u



when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

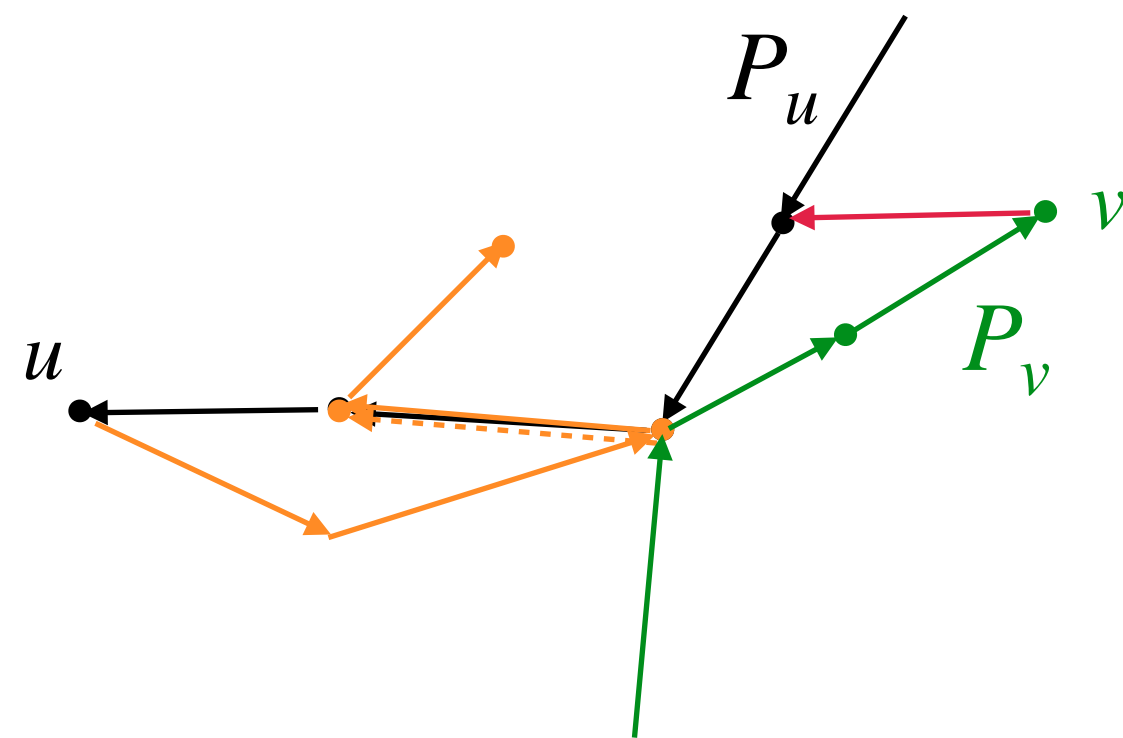
Depends on previous explored edges P_u



when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

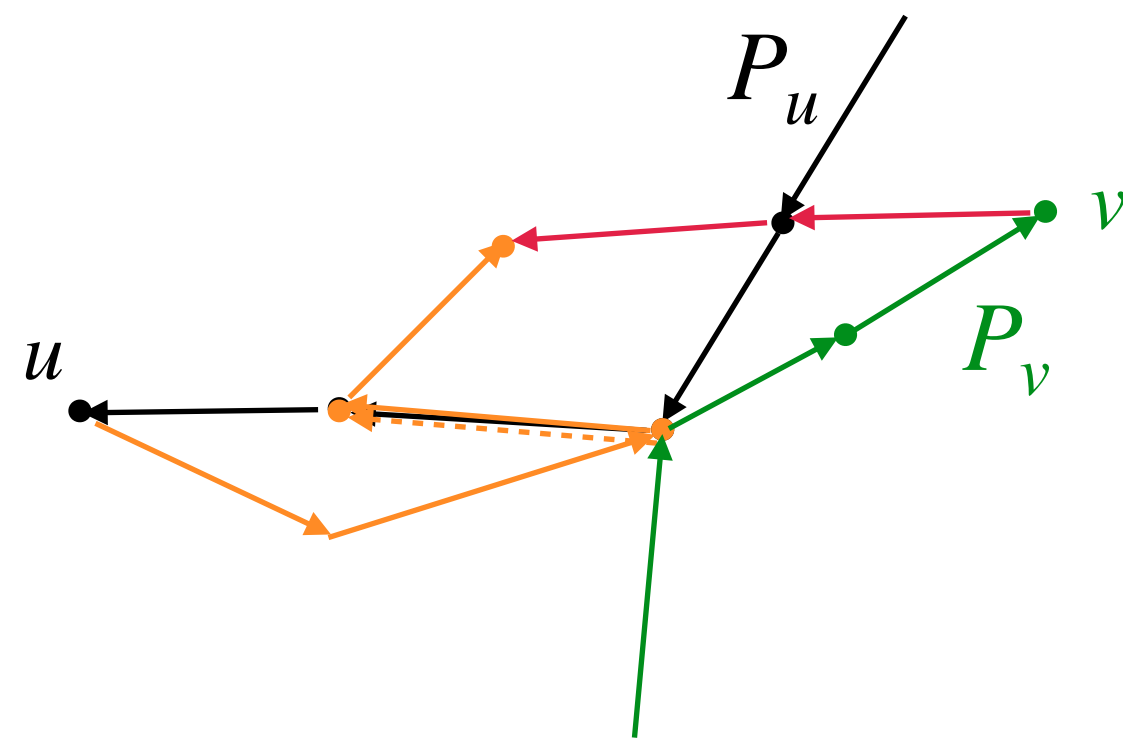
Depends on previous explored edges P_u



when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

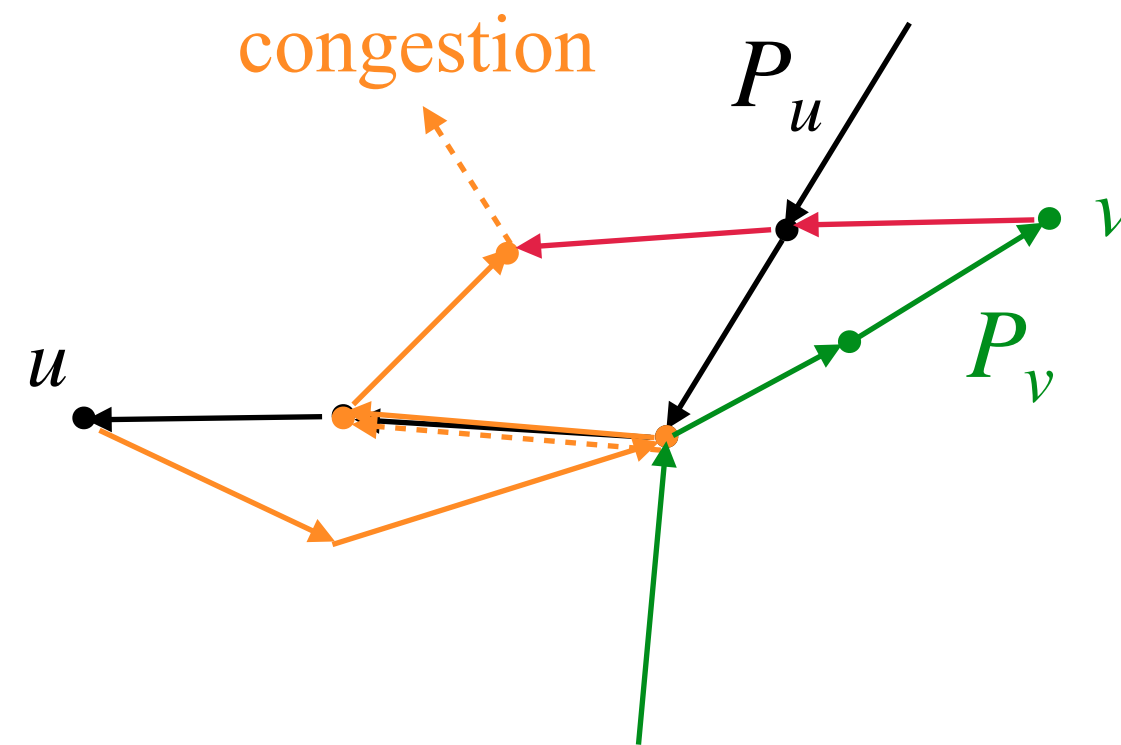
Depends on previous explored edges P_u



when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

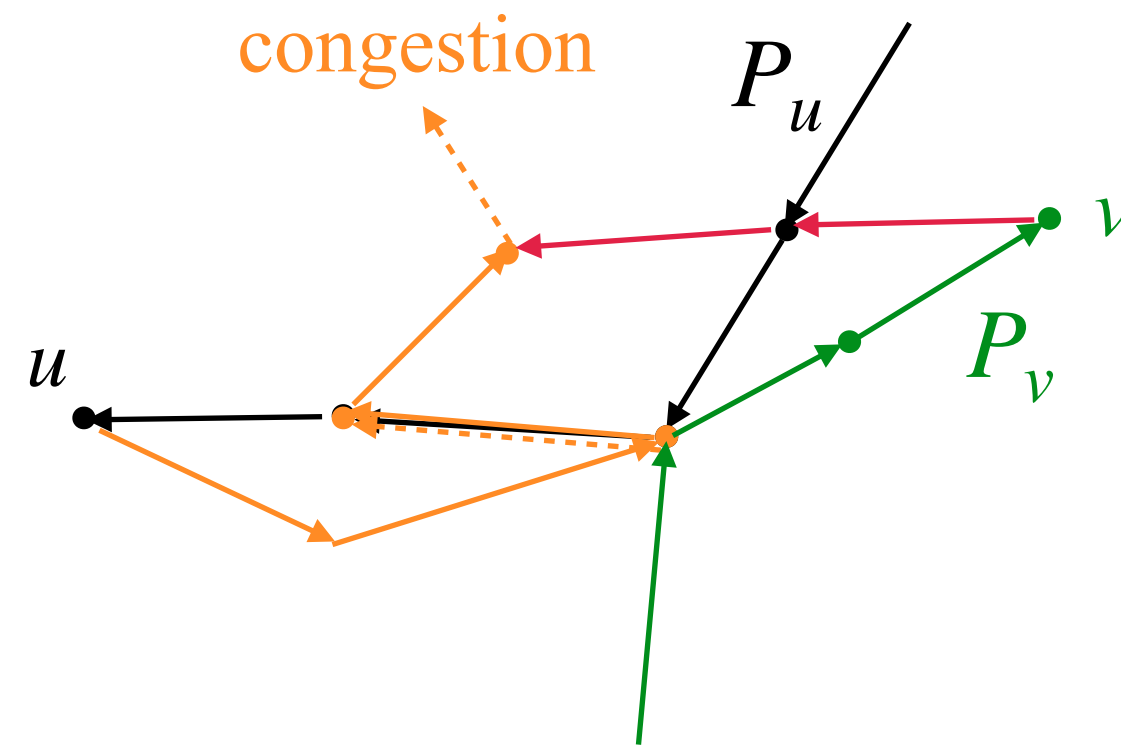
Depends on previous explored edges P_u



when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

Depends on previous explored edges P_u

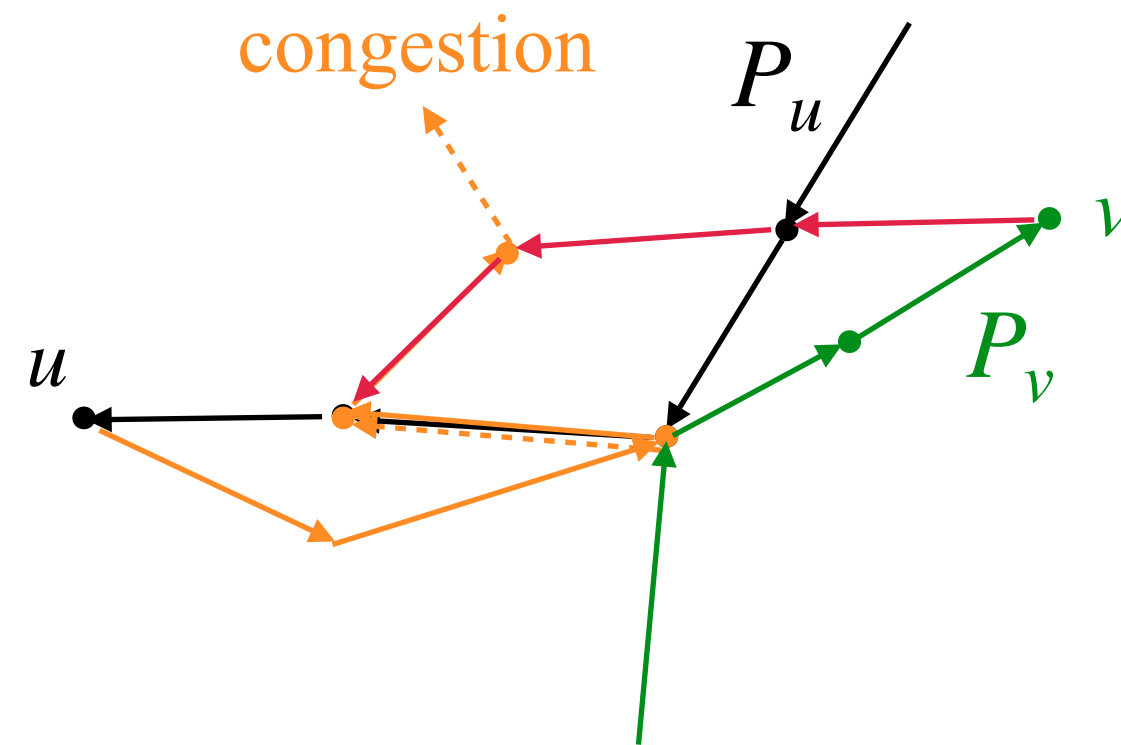


Lemma: by only using edges in P_u , P_v , **augmenting path of u** , **augmenting path of v** , either u can find an augmenting path to v , or v can find an augmenting path to u .

when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

Depends on previous explored edges P_u

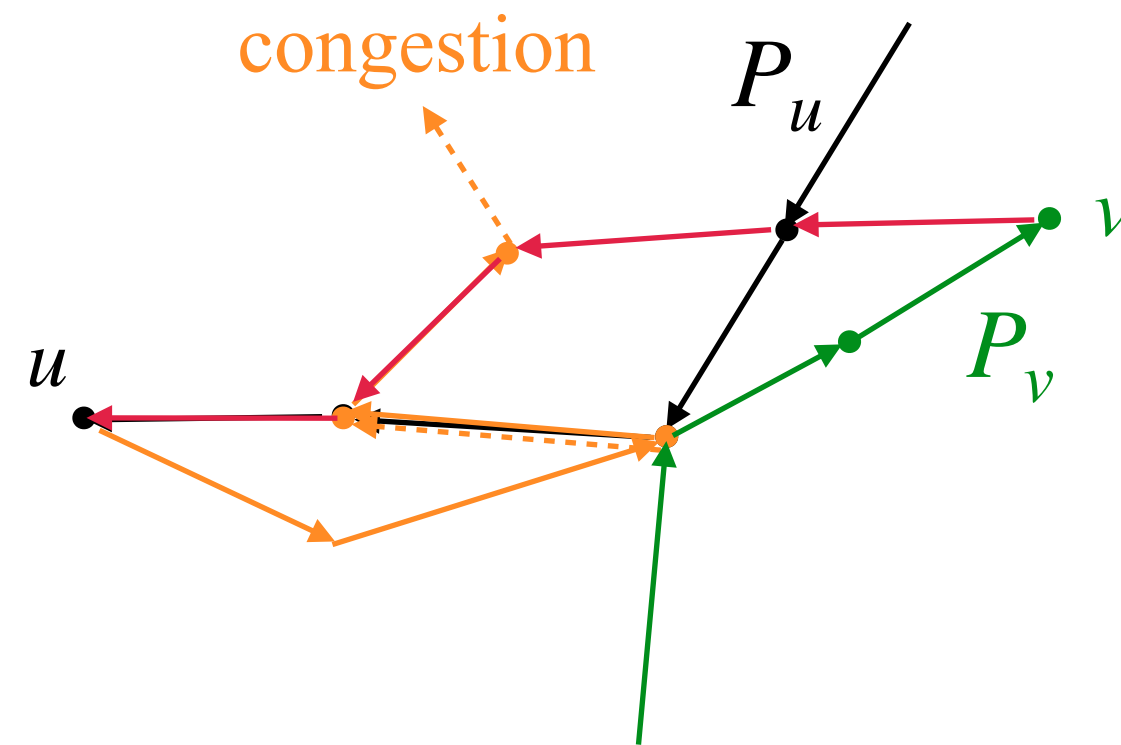


Lemma: by only using edges in P_u , P_v , augmenting path of u , augmenting path of v , either u can find an augmenting path to v , or v can find an augmenting path to u .

when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

Depends on previous explored edges P_u

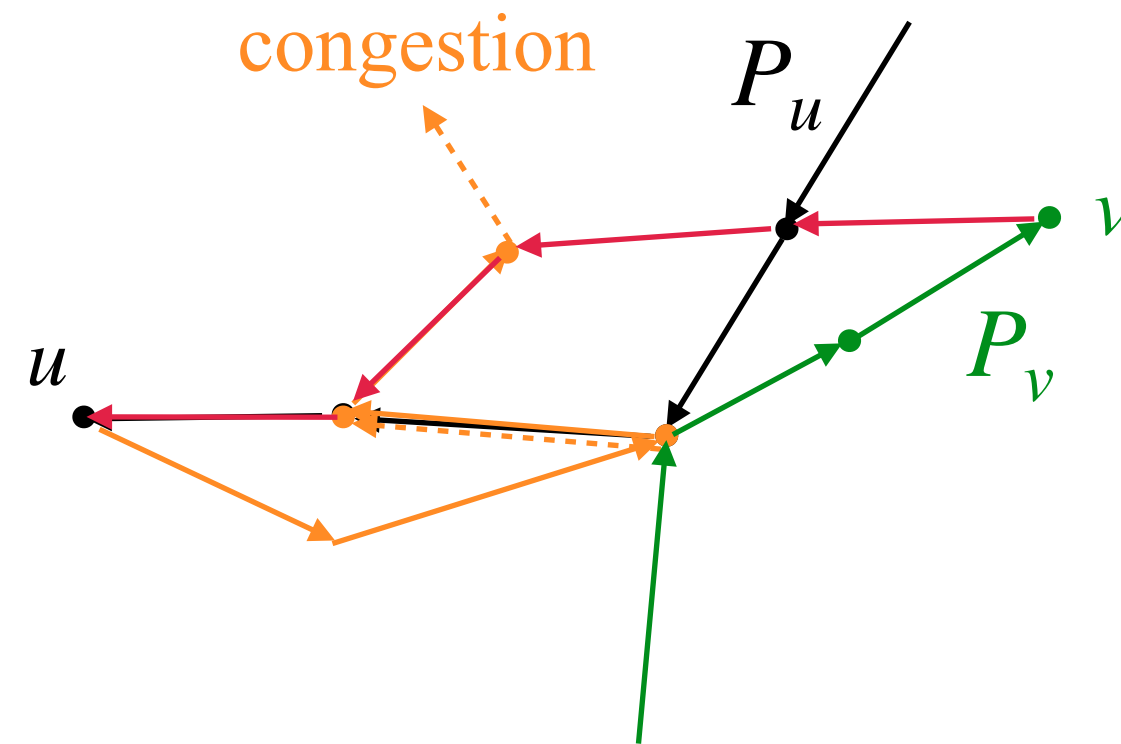


Lemma: by only using edges in P_u , P_v , augmenting path of u , augmenting path of v , either u can find an augmenting path to v , or v can find an augmenting path to u .

when $|L| = o(\sqrt{n})$

- From $n/|L| = \omega(\sqrt{n})$ vertices in R , run k times of DFS to find augmenting paths.

Depends on previous explored edges P_u



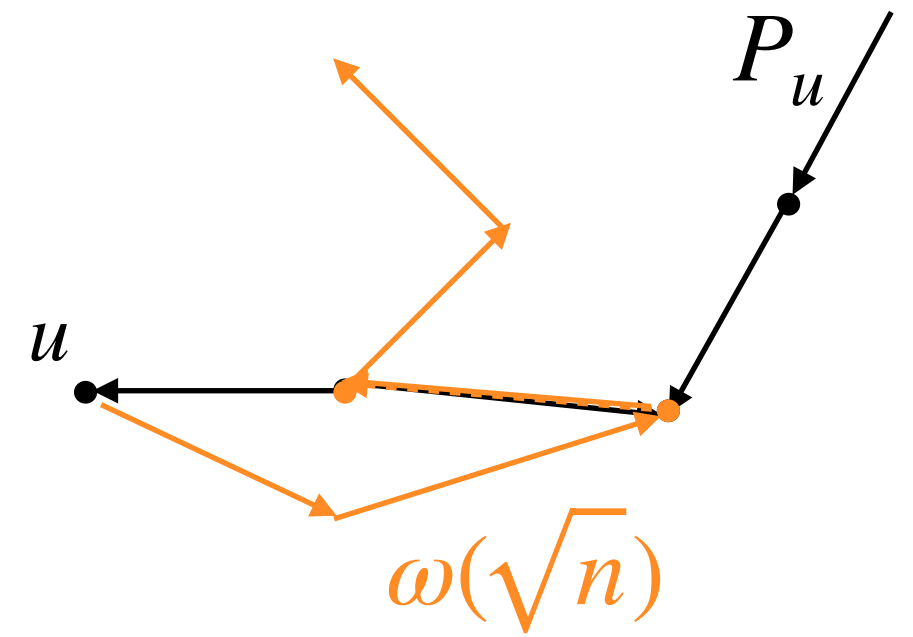
Lemma: by only using edges in P_u , P_v , augmenting path of u , augmenting path of v , either u can find an augmenting path to v , or v can find an augmenting path to u .

Many DFS procedures meet at the same point: pair them up

Outline

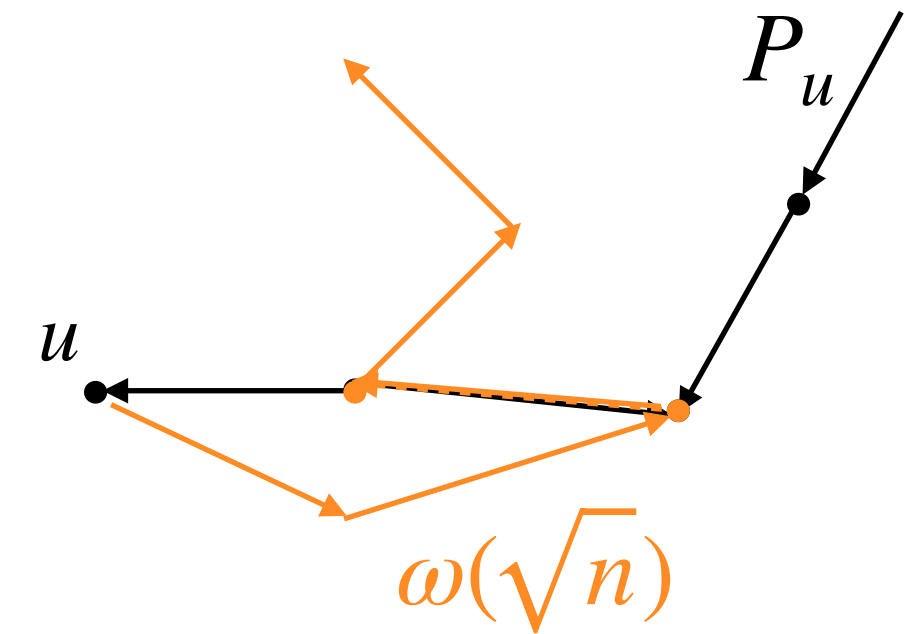
- Sequential algorithm idea.
- Obstacles in CONGEST implementation. $\left\{ \begin{array}{l} \text{Obstacle 1: when } |L| = o(\sqrt{n}), \text{ high congestion!} \\ \text{Obstacle 2: when } |L| = \omega(\sqrt{n}), \text{ high dilation!} \end{array} \right.$
- Solution to obstacle 1
- Solution to obstacle 2

when $|L| = \omega(\sqrt{n})$



when $|L| = \omega(\sqrt{n})$

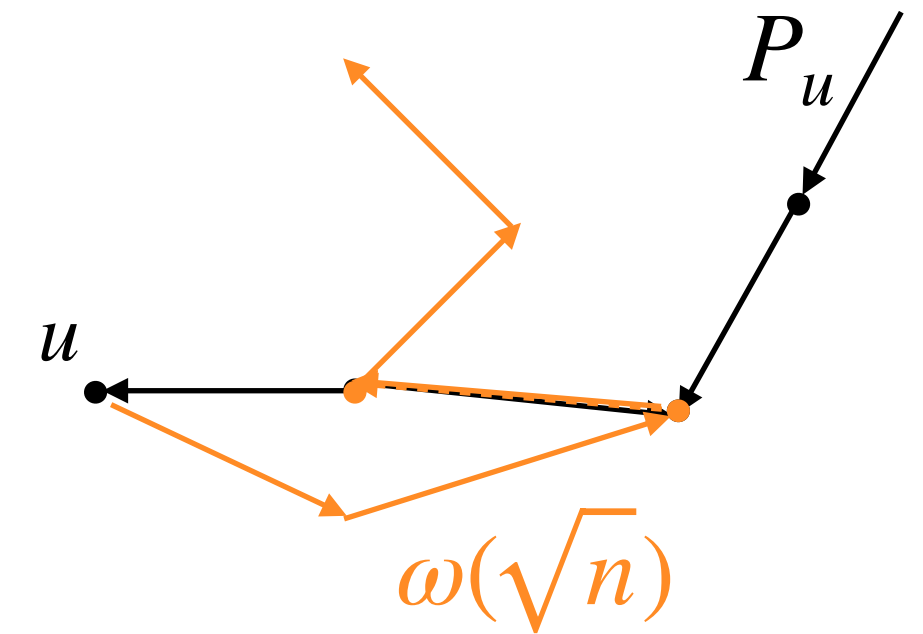
Augmenting path $\rightarrow$ directed path (directed graph defined by P_u)



when $|L| = \omega(\sqrt{n})$

Augmenting path $\rightarrow$ directed path (directed graph defined by P_u)

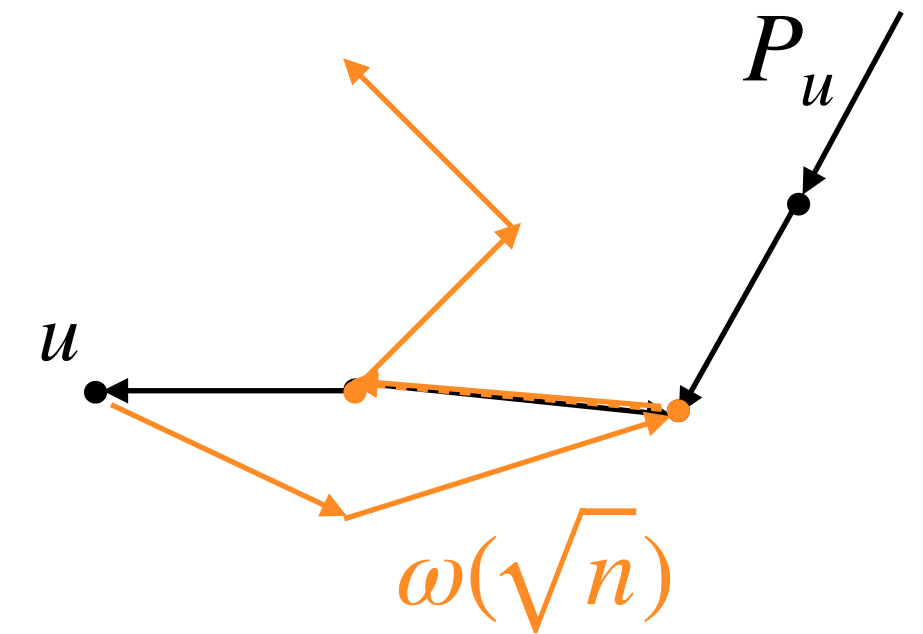
Finding **augmenting path** $\rightarrow$ Reachability $\tilde{O}(n^{1/2} + D + n^{2/5+o(1)}D^{2/5})$



when $|L| = \omega(\sqrt{n})$

Augmenting path $\rightarrow$ directed path (directed graph defined by P_u)

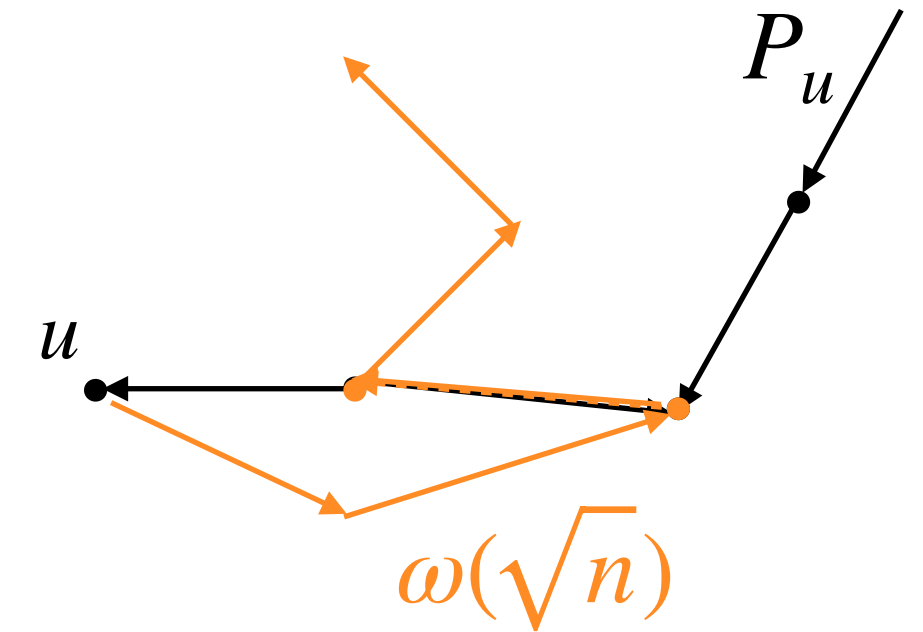
Finding **augmenting path** $\rightarrow$ Reachability $\tilde{O}(n^{1/2} + D + n^{2/5+o(1)}D^{2/5})$
 $> \tilde{O}(n^{1/2} + D)$



when $|L| = \omega(\sqrt{n})$

Augmenting path $\rightarrow$ directed path (directed graph defined by P_u)

Finding **augmenting path** $\rightarrow$ Reachability $\tilde{O}(n^{1/2} + D + n^{2/5+o(1)}D^{2/5})$
 $> \tilde{O}(n^{1/2} + D)$



Our contribution: $k^2 \cdot \tilde{O}(n^{1/2} + D)$ reachability algorithm **on the directed graph defined by P_u**

Where P_u contains $O(k)$ paths.

when $|L| = \omega(\sqrt{n})$

when $|L| = \omega(\sqrt{n})$

Our contribution: $k^2 \cdot \tilde{O}(n^{1/2} + D)$ reachability algorithm **on the directed graph defined by P_u**

Where P_u contains $O(k)$ paths.

when $|L| = \omega(\sqrt{n})$

Our contribution: $k^2 \cdot \tilde{O}(n^{1/2} + D)$ reachability algorithm **on the directed graph defined by P_u**

Where P_u contains $O(k)$ paths.

We have $n/|L|$ vertices in R running reachability in parallel.

when $|L| = \omega(\sqrt{n})$

Our contribution: $k^2 \cdot \tilde{O}(n^{1/2} + D)$ reachability algorithm **on the directed graph defined by P_u**

Where P_u contains $O(k)$ paths.

We have $n/|L|$ vertices in R running reachability in parallel.

Our contribution (new): $k^2 \cdot \tilde{O}(|L|/\sqrt{n})$ congestion, $k^2 \cdot \tilde{O}(\sqrt{n})$ dilation algorithm.

when $|L| = \omega(\sqrt{n})$

Our contribution: $k^2 \cdot \tilde{O}(n^{1/2} + D)$ reachability algorithm **on the directed graph defined by P_u**

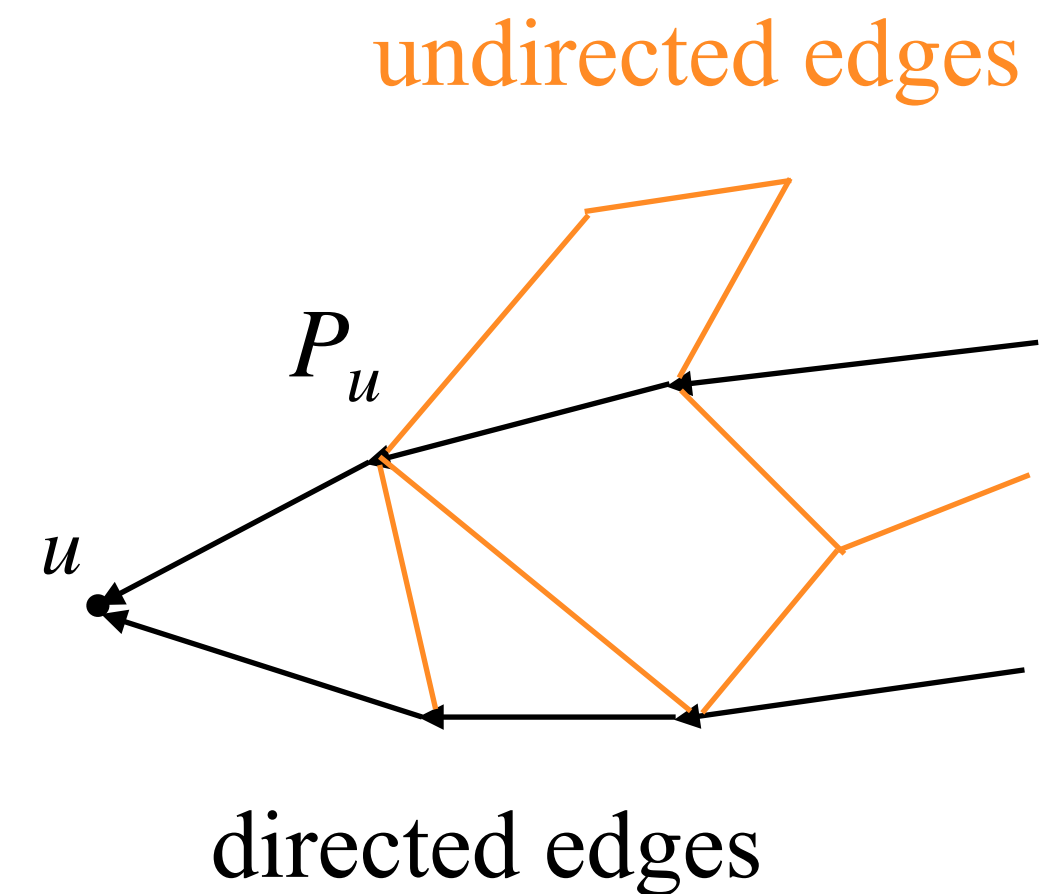
Where P_u contains $O(k)$ paths.

We have $n/|L|$ vertices in R running reachability in parallel.

Our contribution (new): $k^2 \cdot \tilde{O}(|L|/\sqrt{n})$ congestion, $k^2 \cdot \tilde{O}(\sqrt{n})$ dilation algorithm.

Intuitions:

- **Undirected** reachability is easy.
- Directed edges only lie on $O(k)$ paths.



Summary

Our result: a $k^3 \cdot \tilde{O}(\sqrt{n} + D)$ CONGEST algorithm for vertex connectivity

Open Problems

- Better than $\tilde{O}(\sqrt{n} + D)$ algorithm for small k .
- Sub-linear round algorithm for arbitrary k .
- Vertex connectivity in other models: parallel, streaming, ...

Summary

Our result: a $k^3 \cdot \tilde{O}(\sqrt{n} + D)$ CONGEST algorithm for vertex connectivity

Open Problems

- Better than $\tilde{O}(\sqrt{n} + D)$ algorithm for small k .
- Sub-linear round algorithm for arbitrary k .
- Vertex connectivity in other models: parallel, streaming, ...

Thank you!